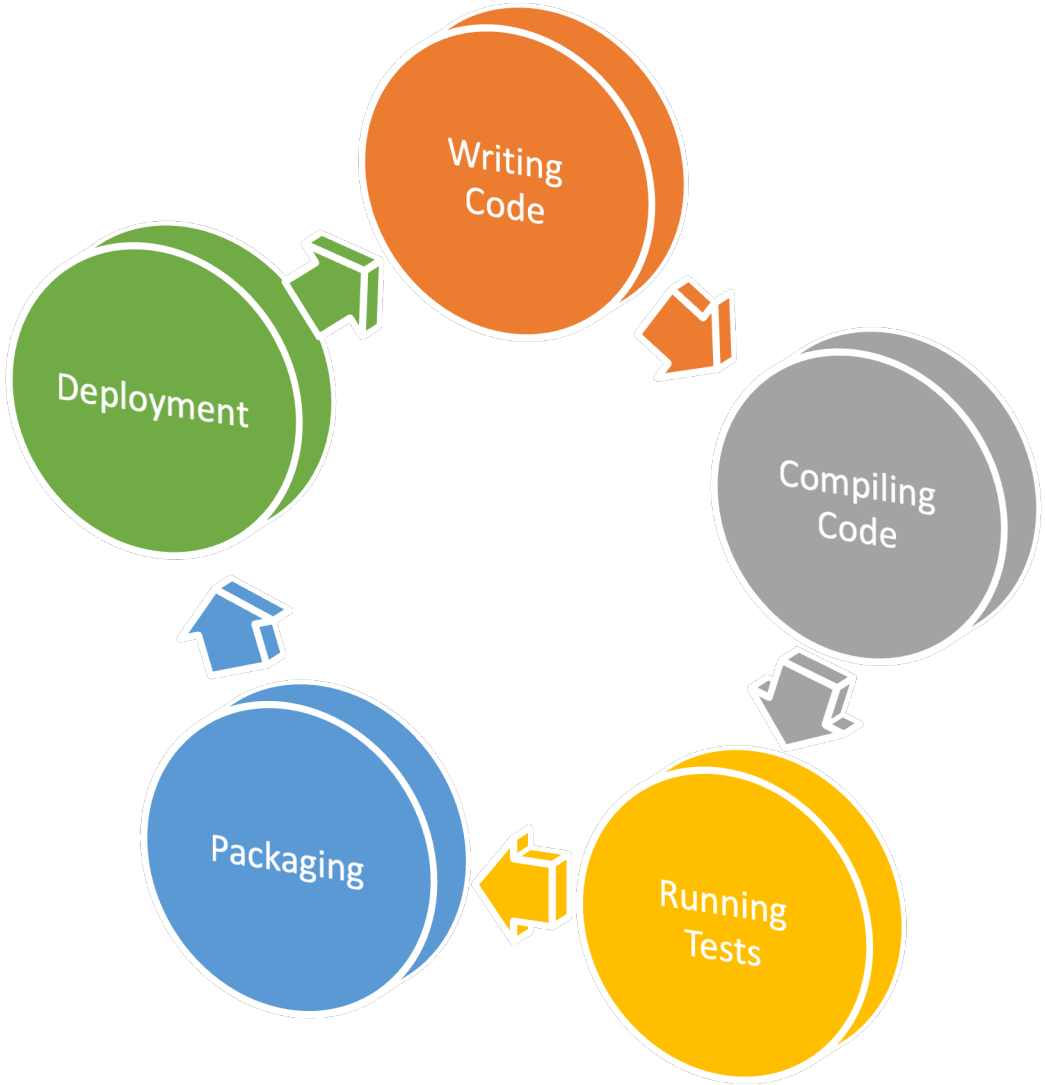

Maven™

Build Automation Tools

Build Automation Tools

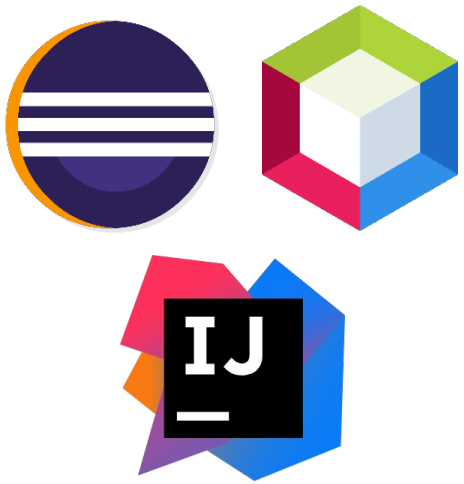
- Traditional software development typically involves writing code, compiling code, running tests, and assembling an archive that finally gets deployed or distributed.
- You might also need to generate some reports and documentation.

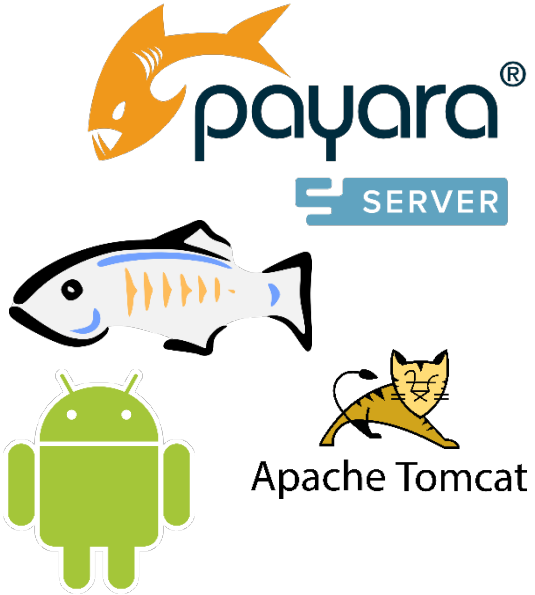
- **Build Automation Tools** helps automating these steps, which helps make builds repeatable and predictable.



Java – Build Automation Tools

What about IDEs !?





Introducing Maven

What is Maven?

Apache Maven is a software
Project Management and **Comprehension Tool**,
that **simplifies**
the building, testing, reporting, and packaging of projects
from a central piece of information,
Based on the concept of a Project Object Model (**POM**).

Maven, a Yiddish word meaning “accumulator of knowledge”

History of Maven

- Maven's initial roots were in the Apache Jakarta Alexandria project that took place in early 2000. It was subsequently used in the Apache Turbine project.
- Like many other Apache projects at that time, the Turbine project had several subprojects, each with its own Ant based build system.

- ***Back then, there was a strong desire for developing a standard way to build projects and to share generated artifacts easily across projects.***
  - ***This desire gave birth to Maven.***

- Maven Versions
  - Version 1.0 ➔ 2004
  - Version 2.0 ➔ 2005
  - Version 3.0 ➔ 2010 *(Currently we're still on 3.x versions of Maven)*

Why Maven is so popular ?

- Maven has become one of the most widely used open-source software programs in enterprises around the world.

- Some of the reasons why Maven is so popular:
  - Standardized Directory Structure
  - Declarative Dependency Management
  - Convention over Configuration
  - Plug-in based architecture
  - Uniform Build Abstraction
  - Archetypes
  - Tools Support
  - Open Source

Survey results of build tool usage

- The State of Developer Ecosystem in 2020 (Java Programming)
  - https://www.jetbrains.com/lp/devecosystem-2020/java/

- Market Share among Developers
  - Maven ➔ 71%
  - Gradle ➔ 48%
  - ANT ➔ 9%

- Maven Alternatives (as a Build Management Tool)
  - Older : Ant + Ivy (very popular in IDE managed projects)
  - Newer : Gradle (very popular in android development)

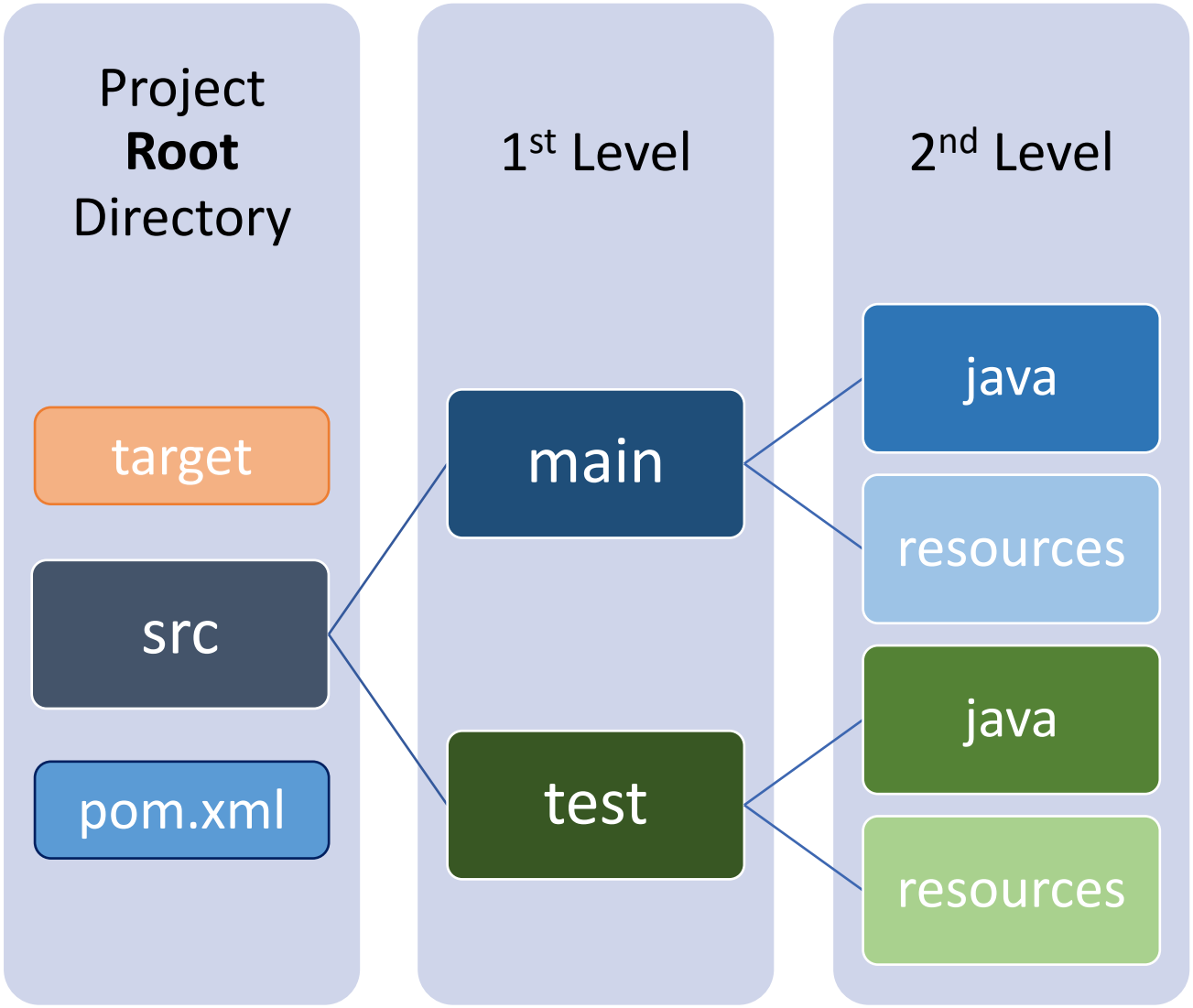
Project Directory Structure

Project Directory Structure

- Maven provides ***conventions***
  and a ***standard directory layout*** for all its projects.

- This standardization provides a uniform build interface,
  and it also makes it easy for developers
  to jump from one project to another.

For Simple Java Projects



The “target” Directory

- The ***target*** directory is the maven ***default output directory***.
- Also Known As “***Project Working Directory***”

- When a project is ***built*** or ***packaged***,
  all the content of the sources, resources and web files will be put inside of it,
  it will be used to construct the artifacts and to run tests.

- You can delete all the target folder with the ***mvn clean*** command.

The “target” Directory Structure

- The “**classes**” folder contains:
  - Compiled Source Files *(that are placed in **src/main/java**)*
  - Resources *(that are placed in **src/main/resources**)*
- The “**test-classes**” folder contains:
  - Compiled Test Source Files *(that are placed in **src/test/java**)*
  - Test Resources *(that are placed in **src/test/resources**)*
- The “**surefire-reports**” folder contains:
  - Test Reports in XML and HTML formats
- The generated project artifact (**.jar**, **.war**, **.ear**, …)
- Folders “**maven-archiver**”, “**maven-status**”
  holds information used by maven during the build.
- Folders “**generated-sources**”, “**generated-test-sources**” holds generated files.

Basic Project Organization

Directory	Description
src/main/java	Application/Library sources
src/main/resources	Application/Library resources
src/main/filters	Resource filter files
src/main/webapp	Web application sources
src/main/config	configuration files, such as tomcat context files, and so on
src/test/java	Test sources
src/test/resources	Test resources
src/test/filters	Test resource filter files
src/it	Integration Tests (primarily for plugins)
src/assembly	Assembly descriptors
src/site	Holds files required during the generation of the project site

Project Object Model

Project Object Model

- This is the "***Central Piece of Information***"

- The "***pom.xml***" file contains all the information needed for
  - Building
  - Testing
  - Reporting
  - Packaging
  - …

Project POM

The minimum requirement for a POM are the following:

- ***project*** - root element
- ***modelVersion*** - version of the POM file itself
- ***groupId*** - the name of the organization this artifact is developed for
- ***artifactId*** - the name of the artifact (project/library/software/application)
- ***version*** - the version of the artifact

```
<project ... >

    <modelVersion> 4.0.0 </modelVersion>

    <groupId> com.example </groupId>
    <artifactId> hello-world </artifactId>
    <version> 1.0-SNAPSHOT </version>

</project>
```

Maven XML Schema Definition

- You can use the Maven POM Schema for Validation
- The Existence of a Schema is Optional

```
<project
      xmlns="http://maven.apache.org/POM/4.0.0"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
            http://maven.apache.org/xsd/maven-4.0.0.xsd" >
```

Maven Artifacts Versioning

It is recommended that Maven projects use the following conventions for versioning:

<major-version> **.** <minor-version> **.** <incremental-version> **-** qualifier

- The ***major***, ***minor***, and ***incremental*** values are ***numeric***.
- The ***qualifier*** can have values such as ***RC***, ***alpha***, ***beta***, and ***SNAPSHOT***.
- Examples that follow this convention are:
  1.0.0, 2.4.5-SNAPSHOT, 3.1.1-RC1, ...

- The ***SNAPSHOT*** qualifier in the project's version ***carries a special meaning***.
- It indicates that the project is in a development stage.
- When a project uses a snapshot dependency, every time the project is built, Maven will fetch and use the latest snapshot artifact.

Skeleton *pom.xml* Contents

```
<project xmlns= “…” xmlns:xsi= “…” xsi:schemaLocation= “…”>

    <modelVersion />
    <groupId />
    <artifactId />
    <version />
    <packaging />
    <name />
    <description />
    <parent />
    <organization />
    <modules />
    <properties />
    <dependencies />
    <dependencyManagement />
    <build />
    <repositories />
    <pluginRepositories />

<project>
```

Maven & JDK9+

- By default, your version of Maven might use an old version of the ***maven-compiler-plugin*** that is not compatible with Java 9 or later versions.

- To target Java 9 or later, you should **at least** use version **3.6.0** of the ***maven-compiler-plugin*** and set the ***maven.compiler.release*** property to the Java release you are targeting (e.g., 9, 10, 11, 12, etc.).

- In the following example, we have configured our Maven project to use version **3.8.1** of ***maven-compiler-plugin*** and target ***Java 11***:

Maven: *Java Project using Java 11*

```
<build>

    <plugins>

        <plugin>
            <artifactId>maven-compiler-plugin</artifactId>
            <version>3.8.1</version>
            <configuration>
                <release>11</release>
            </configuration>
        </plugin>

    </plugins>

</build>
```

Maven: *Java Project using Java 11*

```
<properties>
    <maven.compiler.release> 11 </maven.compiler.release>
</properties>


<build>

    <plugins>

        <plugin>
            <artifactId>maven-compiler-plugin</artifactId>
            <version>3.8.1</version>
        </plugin>

    </plugins>

</build>
```

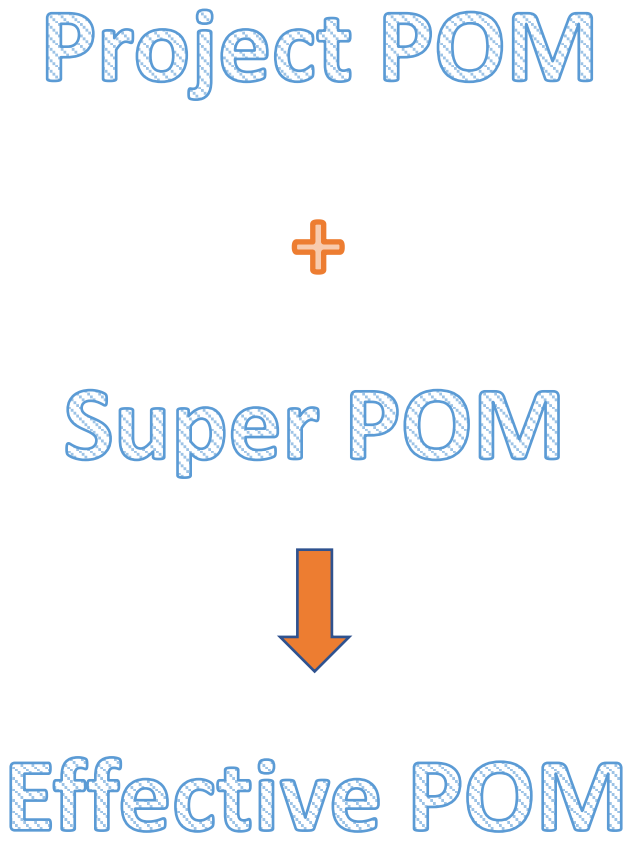
Super-POM & Effective-POM

POM, Super-POM, Effective-POM

- ***The POM*** *(Project POM)*
  - Any Maven Project's POM file defines a groupId, artifactId, and version: the three required coordinates for every project, in addition to any specific configurations or dependencies for that project.

- ***The Super POM***
  - The Super POM is Maven's default POM.
  - All POMs extend the Super POM unless explicitly set, meaning the configuration specified in the Super POM is inherited by the POMs you created for your projects

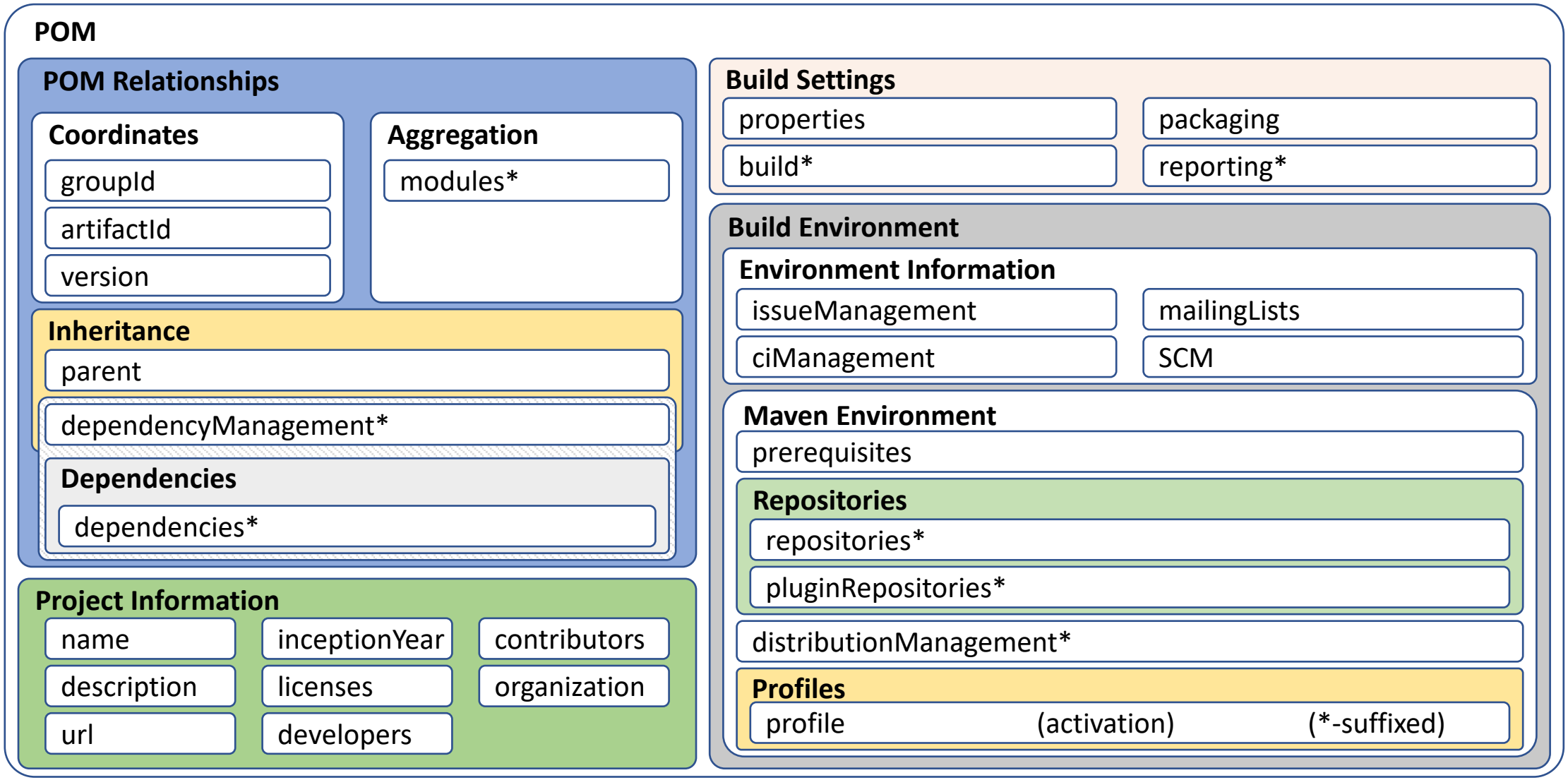
- ***The Effective POM***
  - It is the ***merge*** between The POM and The Super POM.



What is a POM?

- A Project Object Model or POM is the fundamental unit of work in Maven.
- It is an XML file that contains information about the project and configuration details used by Maven to build the project.
- It contains default values for most projects.
- When executing a task or goal, Maven looks for the POM in the current directory.
- It reads the POM, gets the needed configuration information, then executes the goal.
- Some of the configuration that can be specified in the POM are the project dependencies, the plugins or goals that can be executed, the build profiles, and so on.
- Other information such as the project version, description, developers, mailing lists and such can also be specified.

POM


POM
POM Relationships
Coordinates
groupId
artifactId
version
Aggregation
modules*
Inheritance
parent
dependencyManagement*
Dependencies
dependencies*
Project Information
name
inceptionYear
contributors
description
licenses
organization
url
developers
Build Settings
properties
packaging
build*
reporting*
Build Environment
Environment Information
issueManagement
mailingLists
ciManagement
SCM
Maven Environment
prerequisites
Repositories
repositories*
pluginRepositories*
distributionManagement*
Profiles
profile
(activation)
(*-suffixed)

What is a Super POM?

- The Super POM is Maven's default POM.

- All POMs extend the Super POM unless explicitly set, meaning the configuration specified in the Super POM is inherited by the POMs you created for your projects.

- You can see the Super POM for Maven 3.x.x in the Documentation
  - Let’s look at it, to understand maven defaults
  - https://maven.apache.org/ref/3.8.1/maven-model-builder/super-pom.html

Project POM + Super POM

- In the project POM the repositories were not specified.
- If you build your project using this minimal POM,
  it will inherit the repositories configuration in the Super POM.

- Therefore, when Maven sees the dependencies in the project POM,
  it would know that these dependencies will be downloaded from
  https://repo.maven.apache.org/maven2 which was specified in the Super POM.

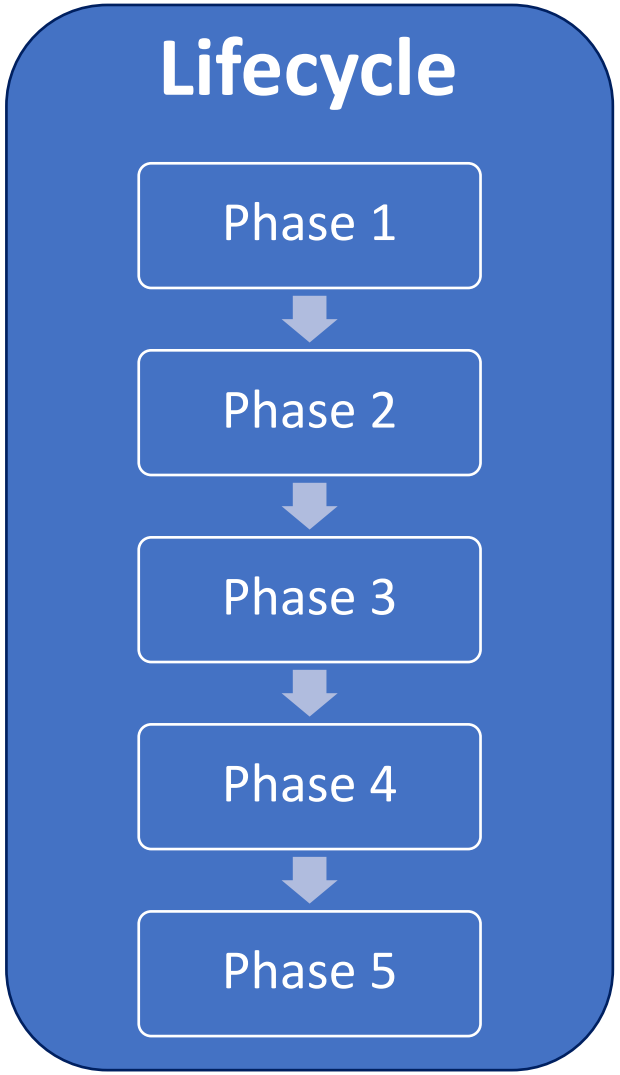
- ***Effective POM = Project POM + Super POM;***

- To view the effective POM, you could use the “help” plugin, or use an IDE
  - **mvn help:effective-pom**

Lifecycles & Phases & Plugins & Goals

What's a Lifecycle ?

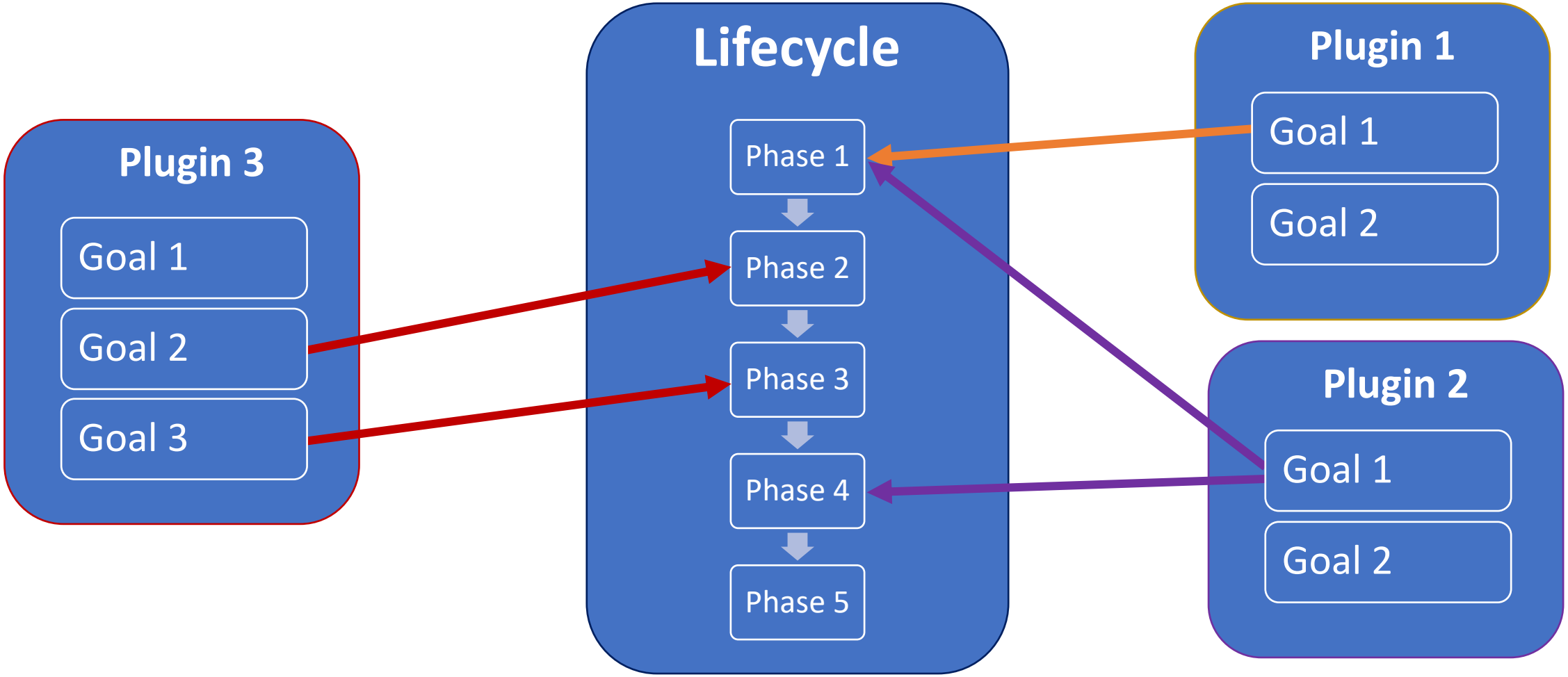
- A lifecycle provides a uniform interface for building and distributing projects.
- A lifecycle constitutes a series of phases that get executed in the same order, independent of the artifact being produced.
- When you execute a phase, all prior phases gets executed first.
- CMD> **mvn** phase-name



What’s a Plug-in ?

- Plugin: contains a set of goals.
- Goal: a piece of functionality (task) to be executed.
  For example: clean, compile, jar, …
- CMD> **mvn** plugin**:**goal

Association between Lifecycle, Phases, Goals, and Plugins



Takeaways

- Maven Lifecycle is an abstract concept and can’t be directly executed.
- Instead, you execute one or more phases.
- In addition to clearly defining the ordering of phases in a lifecycle,
  Maven also automatically executes all the phases prior to a requested phase.

- Several tasks need to be performed in each phase.
  For that to happen, each phase is associated with zero or more goals.
- The phase simply delegates to its associated goals to do the actual work.

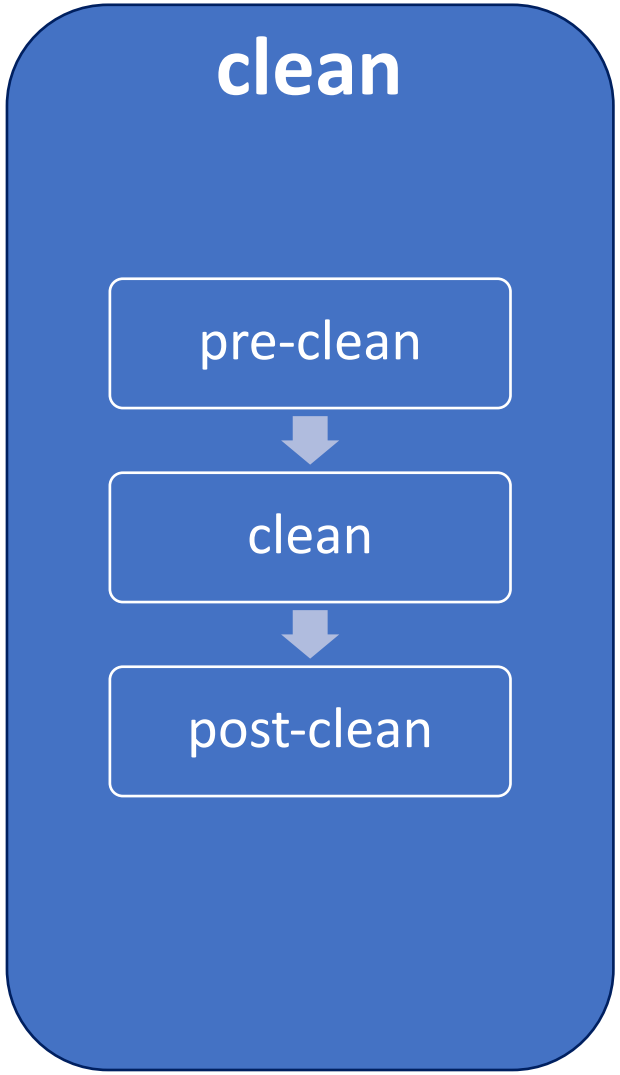
- It is valid for a Maven phase to not have any goals associated with it,
  In that case, Maven will skip the phase execution.
  Such phases serve as placeholders for users and third-party vendors
  to associate their custom-built goals.

Built-In Lifecycles

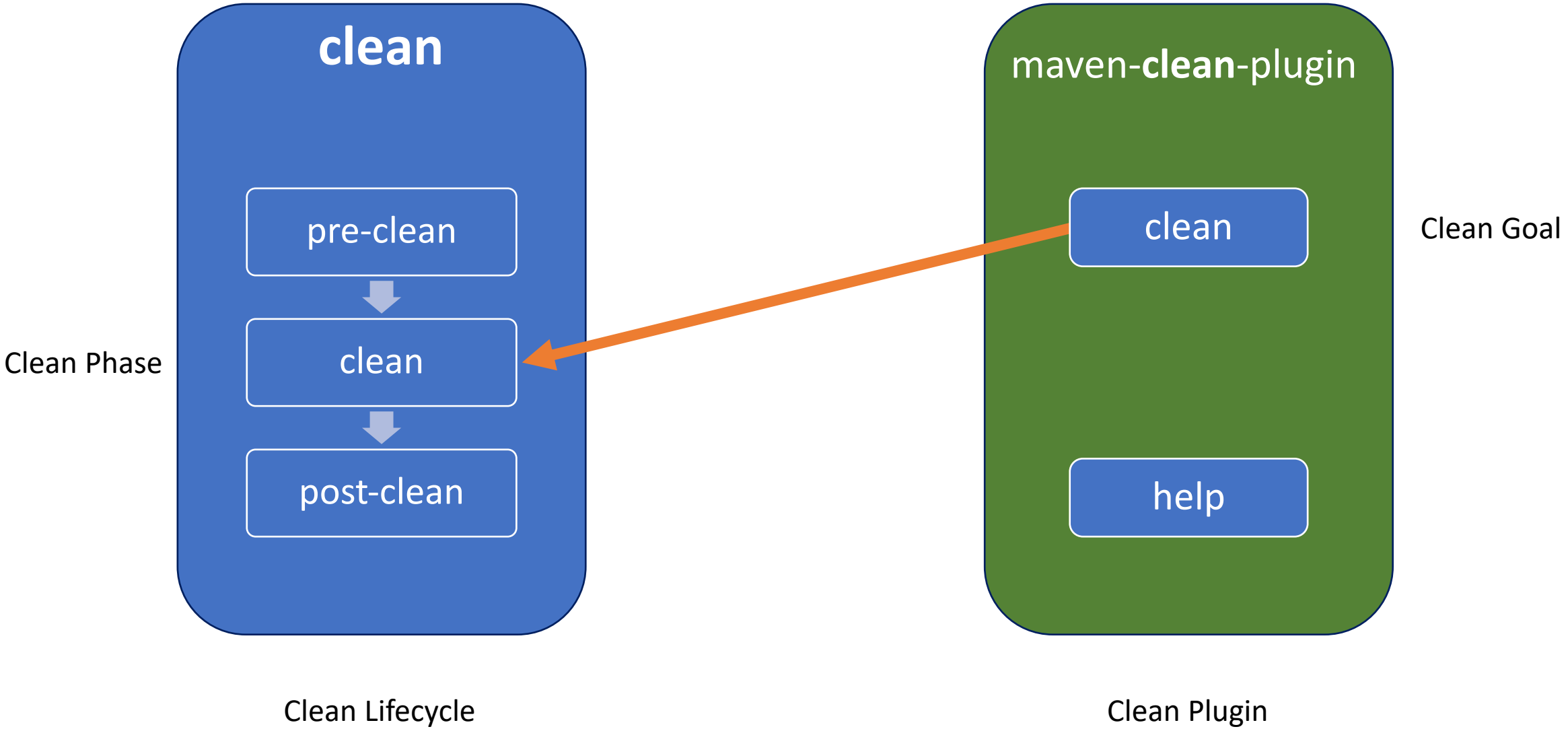
- Every Maven project has the following ***three built-in*** lifecycles:
  - ***default***
    - This lifecycle handles the compiling, packaging, and deployment of a Maven project.
  - ***clean***
    - This lifecycle handles the deletion of temporary files and generated artifacts from the target directory.
  - ***site***
    - This lifecycle handles the generation of documentation and site generation.

The “clean” Lifecycle Phases

- **pre-clean**
  - execute processes needed prior to the actual project cleaning.
- **clean**
  - remove all files generated by the previous build.
- **post-clean**
  - execute processes needed to finalize the project cleaning.

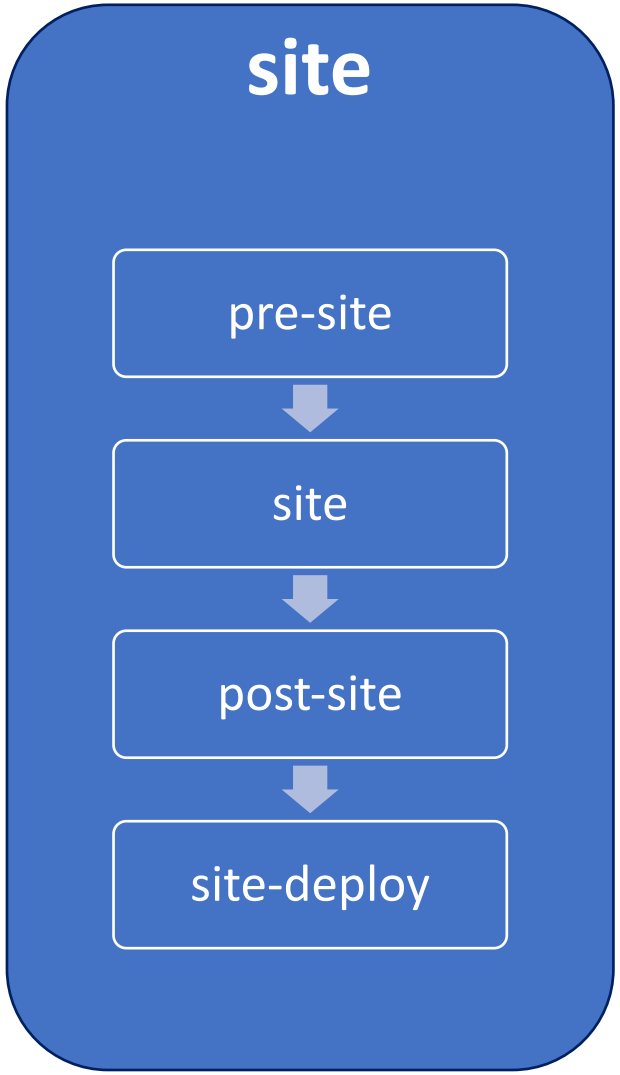


Clean Lifecycle Bindings

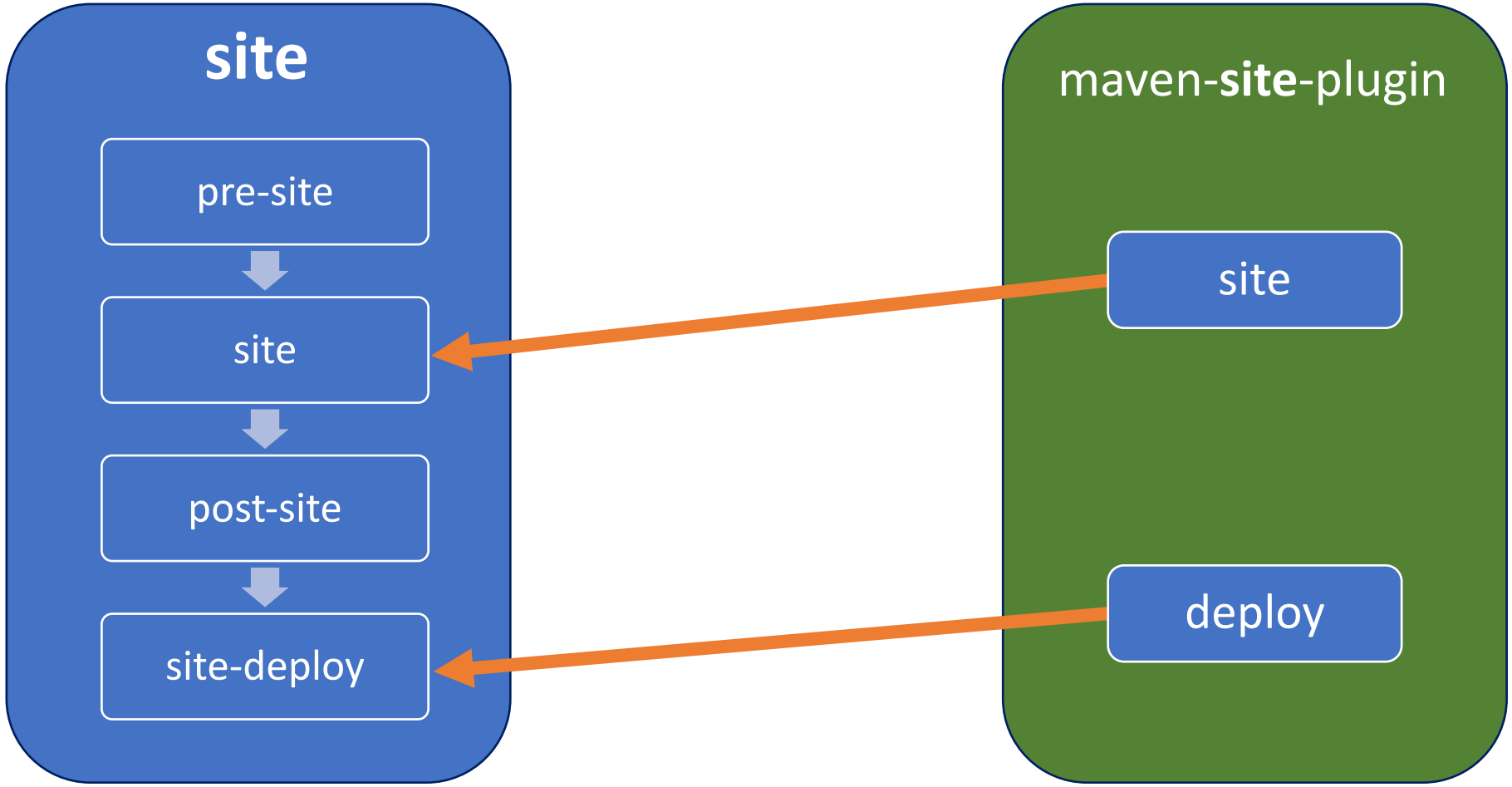


The “site” Lifecycle Phases

- **pre-site**
  - execute processes needed
    prior to the actual project site generation.
- **site**
  - generate the project's site documentation.
- **post-site**
  - execute processes needed to finalize the site generation,
    and to prepare for site deployment.
- **site-deploy**
  - deploy the generated site documentation
    to the specified web server.



Site Lifecycle Bindings

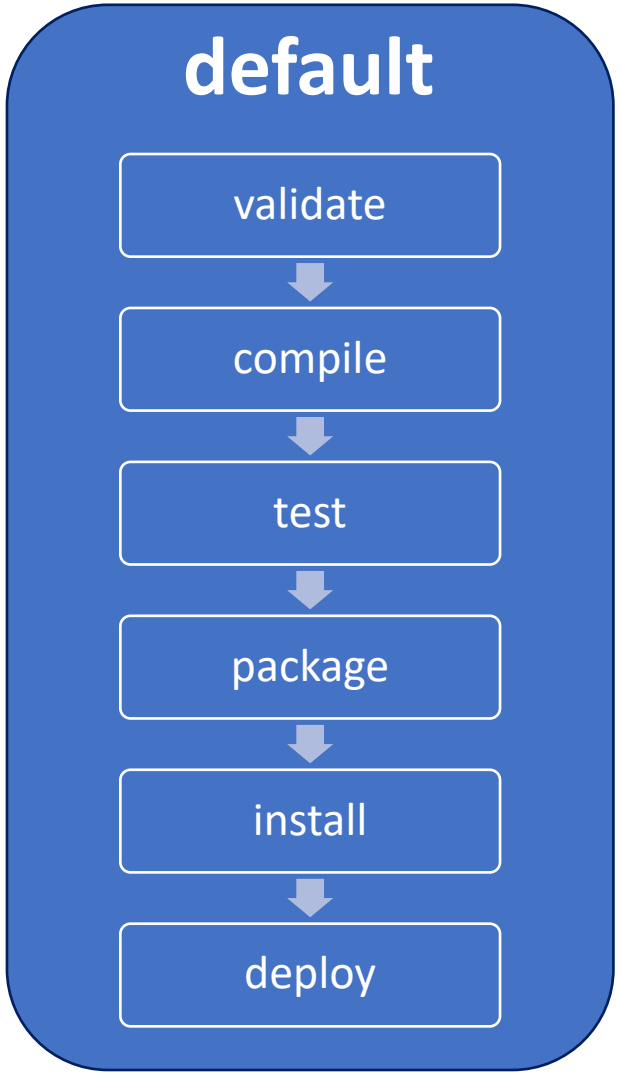


Site Lifecycle

Site Plugin

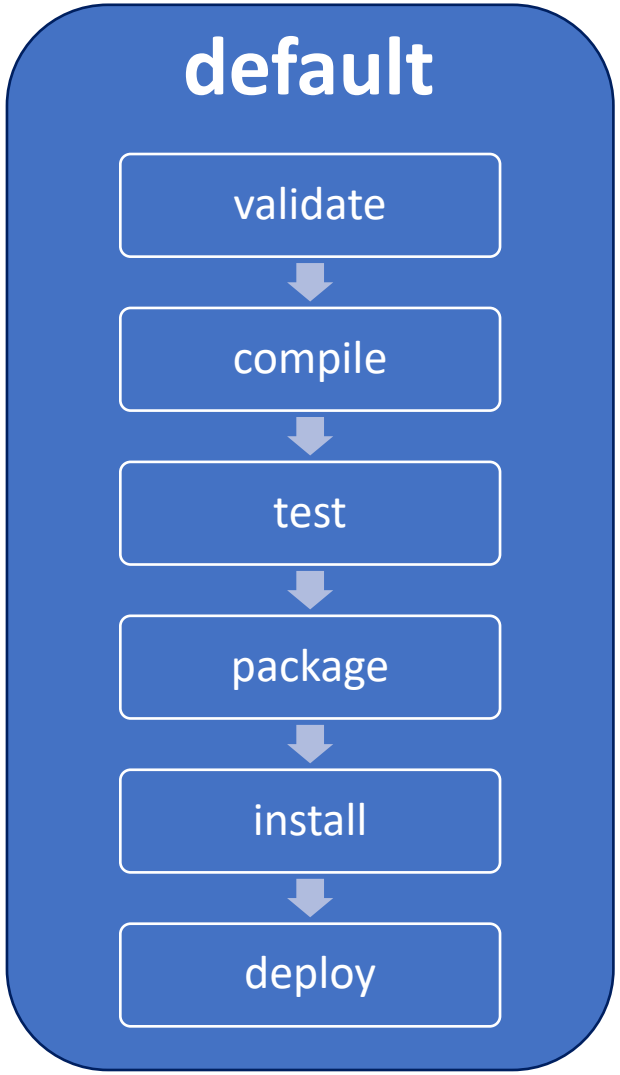
The “default” Lifecycle Phases

- **validate**
  - runs checks to ensure that the project is correct and that all dependencies are downloaded and available.
- **compile**
  - compiles the source code.
- **test**
  - runs unit tests using frameworks. This step doesn’t require that the application be packaged.



The “default” Lifecycle Phases

- **package**
  - assembles compiled code into a distributable format, such as JAR or WAR.
- **install**
  - installs the packaged archive into a local repository.
  - the archive is now available for use by any project running on that machine.
- **deploy**
  - pushes the built archive into a remote repository for use by other teams and team members.

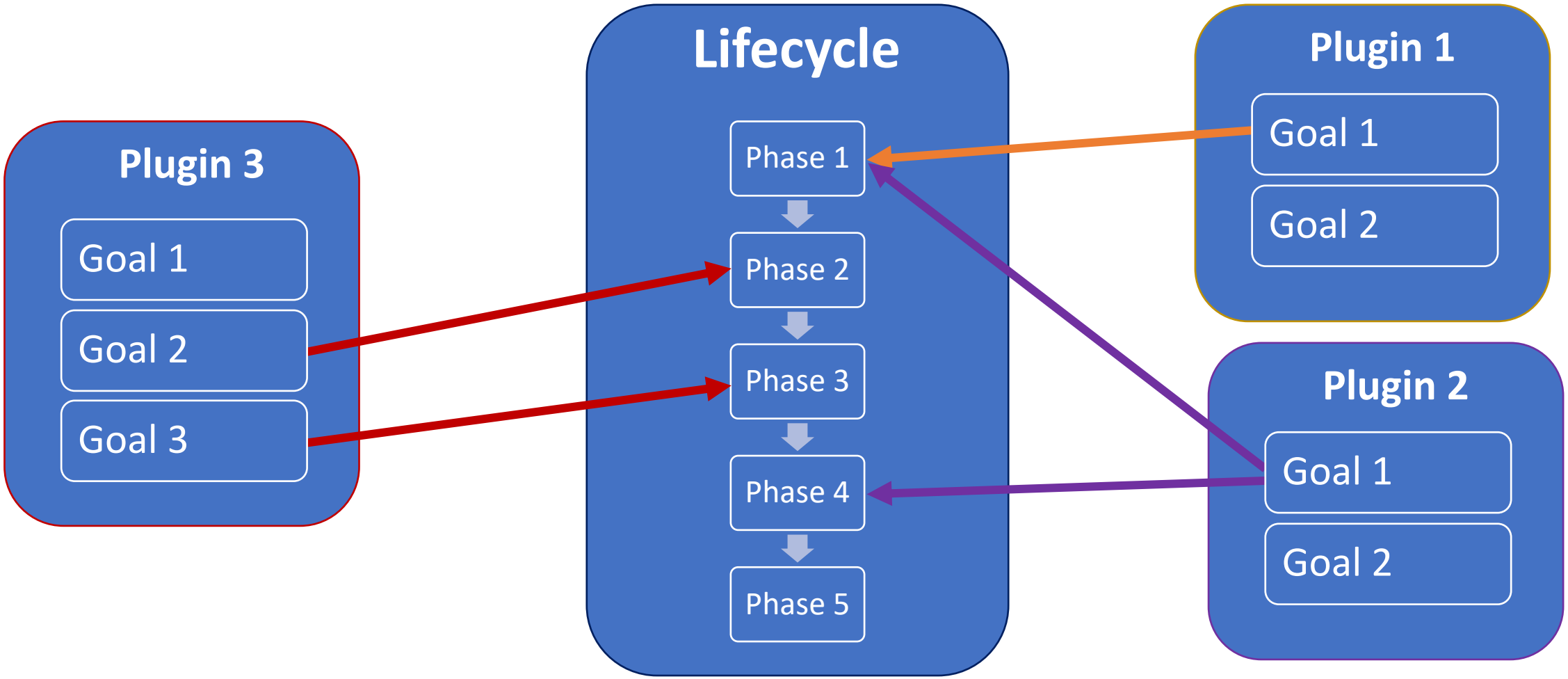


Default Lifecycle in Details

```
<phases>
    <phase>validate</phase>
    <phase>initialize</phase>
    <phase>generate-sources</phase>
    <phase>process-sources</phase>
    <phase>generate-resources</phase>
    <phase>process-resources</phase>
    <phase>compile</phase>
    <phase>process-classes</phase>
    <phase>generate-test-sources</phase>
    <phase>process-test-sources</phase>
    <phase>generate-test-resources</phase>
    <phase>process-test-resources</phase>
    <phase>test-compile</phase>
    <phase>process-test-classes</phase>
    <phase>test</phase>
    <phase>prepare-package</phase>
    <phase>package</phase>
    <phase>pre-integration-test</phase>
    <phase>integration-test</phase>
    <phase>post-integration-test</phase>
    <phase>verify</phase>
    <phase>install</phase>
    <phase>deploy</phase>
</phases>
```

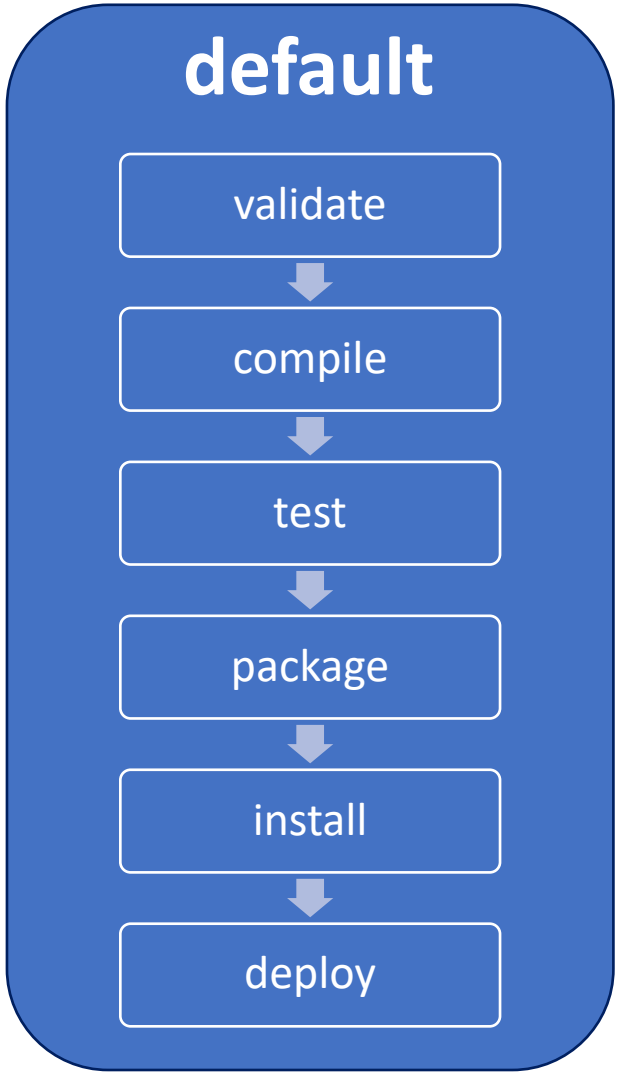
“Default Lifecycle” Bindings

Lifecycle Bindings



The “**default**” Lifecycle Phases

- **validate**: Runs checks to ensure that the project is correct and that all dependencies are downloaded and available.
- **compile**: Compiles the source code.
- **test**: Runs unit tests using frameworks. This step doesn’t require that the application be packaged.
- **package**: Assembles compiled code into a distributable format, such as JAR or WAR.
- **install**: Installs the packaged archive into a local repository. The archive is now available for use by any project running on that machine.
- **deploy**: Pushes the built archive into a remote repository for use by other teams and team members.



Built-in Lifecycle Bindings

- Default Lifecycle Bindings - Packaging **ejb / ejb3 / jar / par / rar / war**

Phase	Plugin : Goal
process-resources	resources:resources
compile	compiler:compile
process-test-resources	resources:testResources
test-compile	compiler:testCompile
test	surefire:test
package	ejb:ejb or ejb3:ejb3 or jar:jar or par:par or rar:rar or war:war
install	install:install
deploy	deploy:deploy

Built-in Lifecycle Bindings

- Default Lifecycle Bindings - Packaging **ear**

Phase	Plugin : Goal
generate-resources	ear:generate-application-xml
process-resources	resources:resources
package	ear:ear
install	install:install
deploy	deploy:deploy

Built-in Lifecycle Bindings

- Default Lifecycle Bindings - Packaging **pom**

Phase	Plugin : Goal
package	
install	install:install
deploy	deploy:deploy

Built-in Lifecycle Bindings

- Default Lifecycle Bindings - Packaging **maven-plugin**

Phase	Plugin : Goal
generate-resources	plugin:descriptor
process-resources	resources:resources
compile	compiler:compile
process-test-resources	resources:testResources
test-compile	compiler:testCompile
test	surefire:test
package	jar:jar and plugin:addPluginArtifactMetadata
install	install:install
deploy	deploy:deploy

Archetypes

Maven Archetypes

- Creating Maven projects manually, is a tedious task,
  creating the folders and creating the pom.xml files from scratch.

- To address this issue, Maven provides ***archetypes***.

- Maven ***archetypes are project templates***
  that allow users to generate new projects easily.

Benefits

- Maven archetypes also provide a great platform to ***share best practices*** and ***enforce consistency*** beyond Maven's standard directory structure.

- For example, an enterprise can create an archetype with the company's branded cascading style sheets (CSS), approved JavaScript libraries, and reusable components.

- Developers using this archetype to generate projects will automatically conform to the company's standards.

Provided Archetypes

- Maven provides hundreds of out-of-the-box archetypes for developers to use.
  - Following the Convention: ***maven-archetype-${archetype-name}***

- Additionally, a lot of open-source projects provide additional archetypes that you can download and use. *(Have no Naming Convention)*

- Maven also provides an ***archetype*** plug-in
  - with goals to create new archetypes
  - and generate projects from existing archetypes

- At the time, there are couple thousand archetypes to choose from.

Provided Archetypes

Archetype ArtifactIds	Description
maven-archetype-archetype	An archetype to generate a sample archetype project.
maven-archetype-j2ee-simple	An archetype to generate a simplifed sample J2EE application.
maven-archetype-mojo	An archetype to generate a sample a sample Maven plugin.
maven-archetype-plugin	An archetype to generate a sample Maven plugin.
maven-archetype-plugin-site	An archetype to generate a sample Maven plugin site.
maven-archetype-portlet	An archetype to generate a sample JSR-268 Portlet.
maven-archetype-quickstart	An archetype to generate a sample Maven project.
maven-archetype-simple	An archetype to generate a simple Maven project.
maven-archetype-site	An archetype to generate a sample Maven site which demonstrates some of the supported document types like APT, XDoc, and FML and demonstrates how to i18n your site.
maven-archetype-site-simple	An archetype to generate a sample Maven site.
maven-archetype-webapp	An archetype to generate a sample Maven Webapp project.

Creating a Project from an Archetype

- Plugin : **archetype**
- Goal : **generate**

```
mvn archetype:generate
        -DgroupId=com.mycompany.app
        -DartifactId=my-app
        -DarchetypeArtifactId=maven-archetype-quickstart
        -DarchetypeVersion=1.4
        -DinteractiveMode=false
```

Creating a Project from an Archetype

- Plugin : **archetype**
- Goal : **generate**

```
mvn archetype:generate
        -DarchetypeArtifactId=maven-archetype-webapp
        -DarchetypeVersion=1.4
```

This will run in ***interactive mode***,

it'll prompt you about what's the groupId, artifactId, version

of the artifact you want to create.

Generating a Hello World Project

- The archetype plugin can be used to create a “Hello World” Project.
- Plugin : archetype
- Goal : generate

```
mvn archetype:generate
        -DgroupId=com.mycompany.app
        -DartifactId=my-app
        -DinteractiveMode=false
```

Dependencies

The OLD Way of Adding Dependencies

Consider the scenario where you want to use the library Log4J for your application logging.

To accomplish this

1. You would go to the Log4J download page
2. Download the JAR file
3. Put it in your project's lib folder or add it to the project's class path

Problems with this OLD Approach

1. The JAR file you downloaded might depend on a few other libraries.
   You would now have to hunt down all those dependencies
   (and their dependencies) and add them to your project.

2. When the time comes to upgrade the JAR file,
   you need to start the process all over again.

3. You need to add JAR files to source control along with your source code
   so that your projects can be built on a computer other than your own.
   This increases project size, checkout, and build time.

4. Sharing JAR files across teams within your organization becomes difficult.

What Maven Did ?

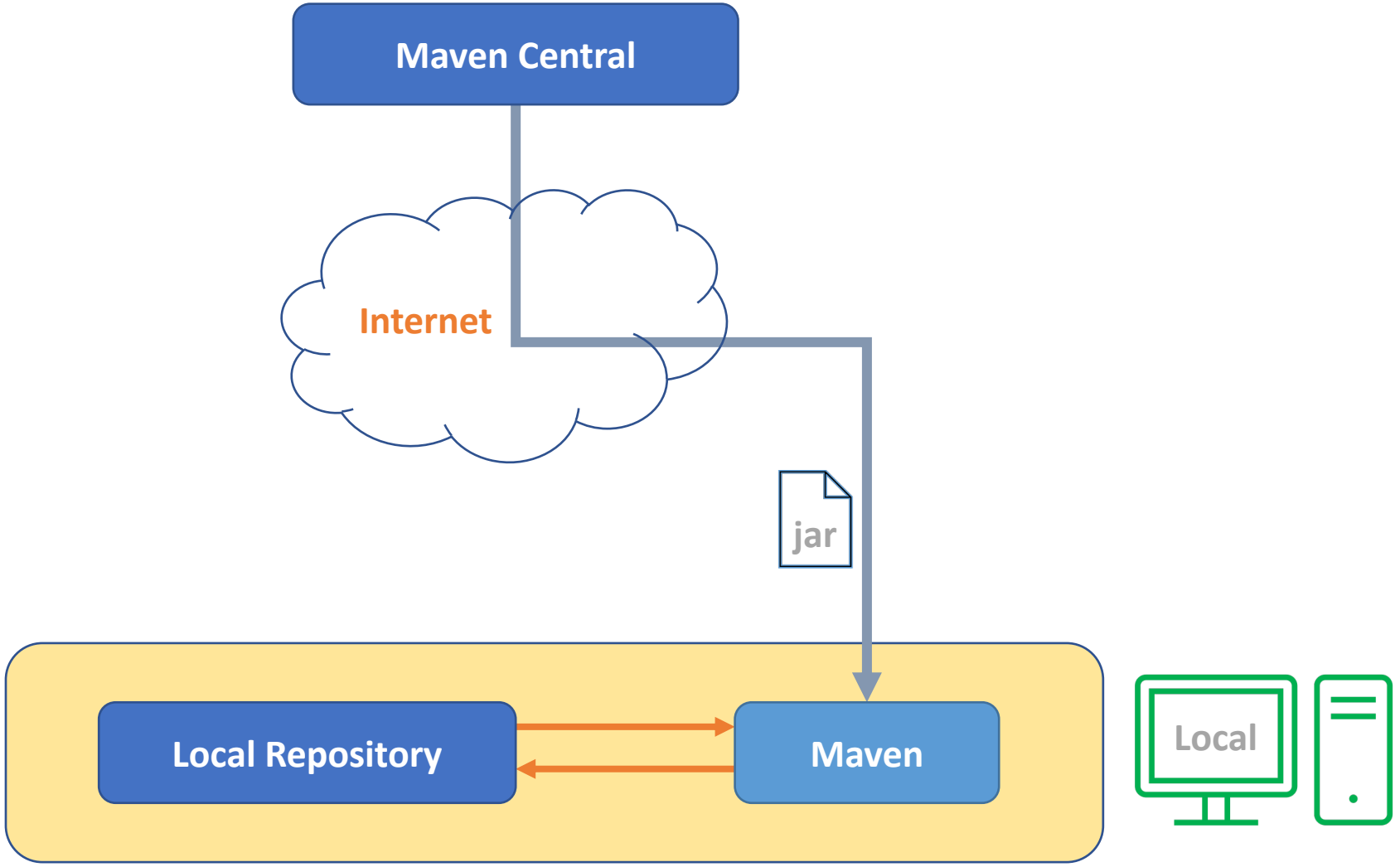
To address these problems,

Maven provides ***declarative dependency management***.

With this approach, you declare your project's dependencies in an external file called **pom.xml**.

Maven will automatically download those dependencies and hand them over to your project for the purpose of building, testing, or packaging.

Maven Dependencies



How it works ?

- When you run your Maven project for the first time,
  Maven connects to the network and downloads artifacts and related metadata
  from remote repositories.

- The default remote repository is called **Maven Central**,
  and it is located at *repo.maven.apache.org* and *uk.maven.org*.
- Maven places a copy of these downloaded artifacts in its local repository.

- In subsequent runs, Maven will look for an artifact in its local repository;
  and upon not finding the artifact,
  Maven will attempt to download it from remote repository.

Dependency Declaration

Dependency Identification

Maven dependencies are typically archived as JAR, WAR, EAR, RAR, EJB, ZIP, ...

Each Maven dependency is uniquely identified using the following
group, artifact, and version (GAV) coordinates:

groupId: Identifier of the organization or group that is responsible for this artifact (dependency).
Examples include org.hibernate, log4j, org.springframework and com.companyname.

artifactId: Identifier of the artifact (dependency).
This must be unique among the projects using the same groupId.
Examples include hibernate-tools, log4j, springcore, and so on.

version: Indicates the version number of the dependency.
Examples include 1.0.0, 2.3.1-SNAPSHOT, and 5.4.2.Final.

type: Indicates which packaging is needed of that artifact.
Examples include JAR, WAR, POM, and EAR.

scope: Indicates the scope of the dependency.
Examples include compile, runtime, provided, test, system, import.

Declaring Dependencies

Dependencies are declared in **pom.xml** file
using the ***<dependencies>*** tag as shown in the following:

<dependencies>

```
<dependency>
    <groupId>org.hibernate</groupId>
    <artifactId>hibernate-tools</artifactId>
    <version>5.4.2.Final</version>
</dependency>
```

</dependencies>

Dependency Scopes

Dependency Scopes

- Consider a Java project that uses JUnit for its unit testing.
- The JUnit JAR file you included in your project is only needed during testing.
- You really don’t need to bundle the JUnit JAR in your final production archive.

- Similarly, consider the MySQL database driver, mysql-connector-java.jar file.
- You need the JAR file when you are running the application inside a container such as Tomcat but not during code compilation or testing.

- Maven uses the concept of ***scope***, which allows you to specify when and where you need a particular dependency.
- Maven provides the following ***six scopes***
  - compile, provided, runtime, test, system, import

Dependency Scope

compile: Dependencies with the compile scope are available in the class path in ***all phases*** on a project build, test, and run. *(This is the default scope)*

```
<dependency>
        <groupId> org.slf4j </groupId>
        <artifactId> slf4j-api </artifactId>
        <version> 1.7.30 </version>
        <scope> compile </scope>
</dependency>
```

Dependency Scope

provided: Dependencies with the provided scope are available in the class path during the ***build*** and ***test*** phases.

➔ They don't get bundled within the generated artifact.

```
<dependency>
        <groupId> javax.servlet </groupId>
        <artifactId> javax.servlet-api </artifactId>
        <version> 4.0.1 </version>
        <scope> provided </scope>
</dependency>
```

Dependency Scope

runtime: Dependencies with the runtime scope
are ***not available in the class path during the build phase***.

➔ Instead, they get bundled in the generated artifact and are ***available during runtime***.

```
<dependency>
        <groupId> mysql </groupId>
        <artifactId> mysql-connector-java </artifactId>
        <version> 8.0.21 </version>
        <scope> runtime </scope>
</dependency>
```

Dependency Scope

test: Dependencies with the test scope are available during the ***test*** phase.

JUnit and TestNG are good examples of dependencies with the test scope.

```
<dependency>
        <groupId> org.junit.jupiter </groupId>
        <artifactId> junit-jupiter-api </artifactId>
        <version> 5.6.2 </version>
        <scope> test </scope>
</dependency>
```

Dependency Scope

system: Dependencies with the system scope are ***not retrieved from the repository***.

➔ Instead, a hard-coded path to the file system is specified from which the dependencies are used.

```
<dependency>
        <groupId> com.oracle </groupId>
        <artifactId> ojdbc </artifactId>
        <version> 8 </version>
        <scope> system </scope>
        <systemPath> D:/projects/ojdbc8.jar </systemPath>
</dependency>
```

Dependency Scope

import: The import scope is applicable for **.pom** file dependencies only. It allows you to ***include dependency management information*** from a remote **.pom** file.

This scope is only supported on a dependency of type ***pom*** in the ***<dependencyManagement>*** section.

It indicates the dependency is to be replaced with the effective list of dependencies in the specified POM's ***<dependencyManagement>*** section.

Since they are replaced, dependencies with a scope of import do not actually participate in limiting the transitivity of a dependency.

Transitive Dependencies

Transitive Dependencies

- Dependencies declared in your project's **pom.xml** file often have their own dependencies.

- Such dependencies are called ***transitive dependencies***.

- Notice that transitive dependencies can have their own dependencies.

- As you might imagine, this can quickly get complex, especially when multiple direct dependencies pull different versions of the same JAR file.

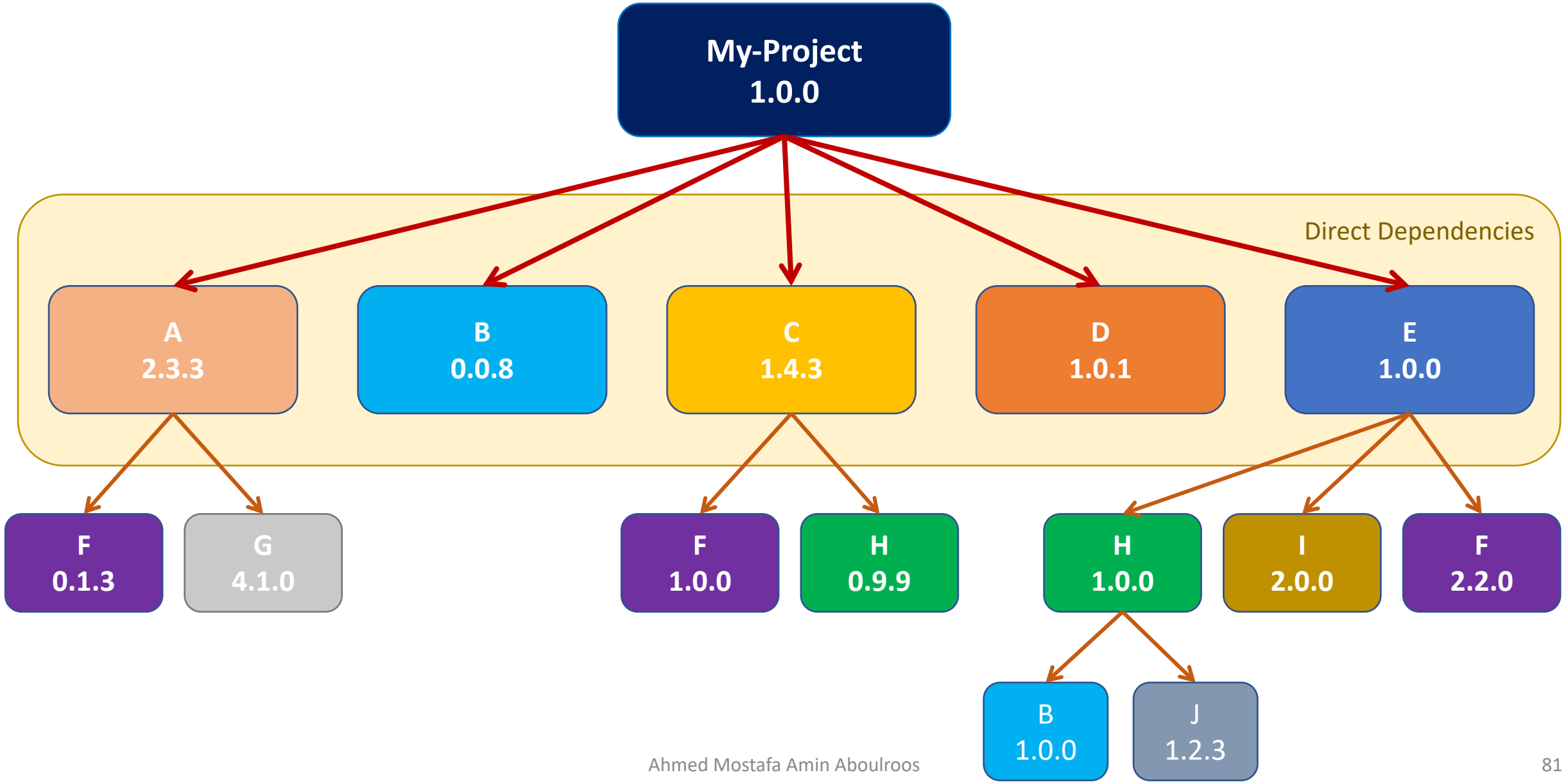
Dependency Mediation

Dependency Mediation

- Maven uses a technique known as “Dependency Mediation” to resolve version conflicts.

- Simply stated, dependency mediation allows Maven to pull the dependency that is ***closest to the project in the dependency tree***.

Dependency Mediation



Visualize Project Dependency Tree

- Although highly useful, transitive dependencies can cause problems and unpredictable side effects, as you might end up including unwanted JAR files or older versions of JAR files.

- Maven provides a handy dependency plug-in
  that allows you to visualize project dependency tree.

  - CMD> **mvn** dependency**:**tree

- Some IDEs have a graphical visualization tools for dependencies.

Dependency Exclusion

Dependency Exclusion

<dependencies>

```
<dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>4.12</version>
    <exclusions>
        <exclusion>
            <groupId>org.hamcrest</groupId>
            <artifactId>hamcrest</artifactId>
        </exclusion>
    </exclusions>
</dependency>
```

</dependencies>

Optional Dependencies

Optional Dependencies

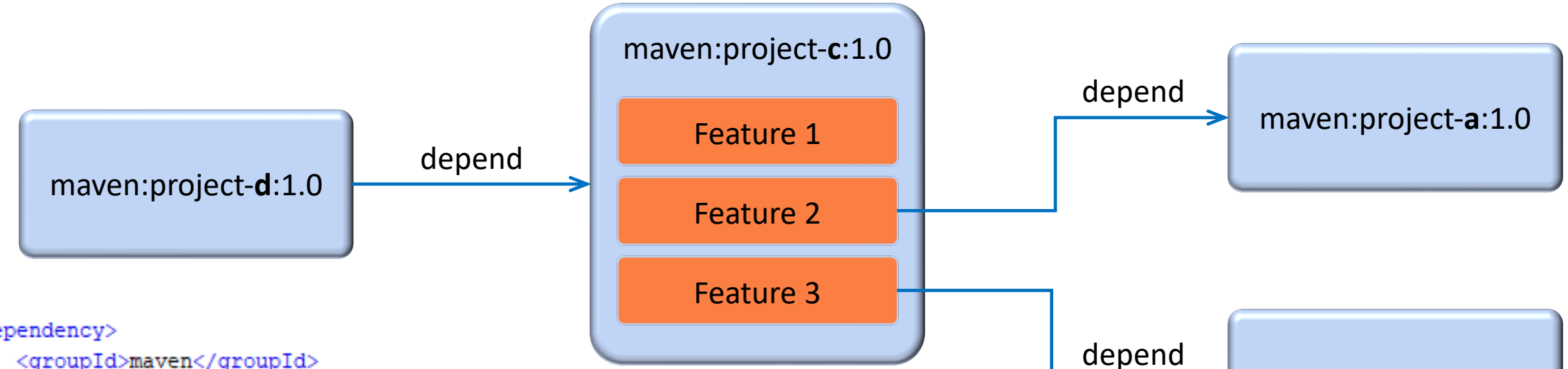
- Optional dependencies are used when it's not possible *(for whatever reason)* to split a project into sub-modules.
- ***The idea is that some of the dependencies are only used for certain features in the project and will not be needed if that feature isn't used.***

- Ideally, such a feature would be split into a sub-module that depends on the core functionality project.
- This new subproject would have only non-optional dependencies, since you'd need them all if you decided to use the subproject's functionality.

- However, since the project cannot be split up *(again, for whatever reason)*, these dependencies are declared optional.
- If a user wants to use functionality related to an optional dependency, they have to ***redeclare*** that optional dependency in their own project.

Optional Dependencies

- Project D do NOT depend on A, B unless you declare A & B Explicitly




```
<dependency>
    <groupId>maven</groupId>
    <artifactId>project-c</artifactId>
    <version>1.0</version>
</dependency>
```

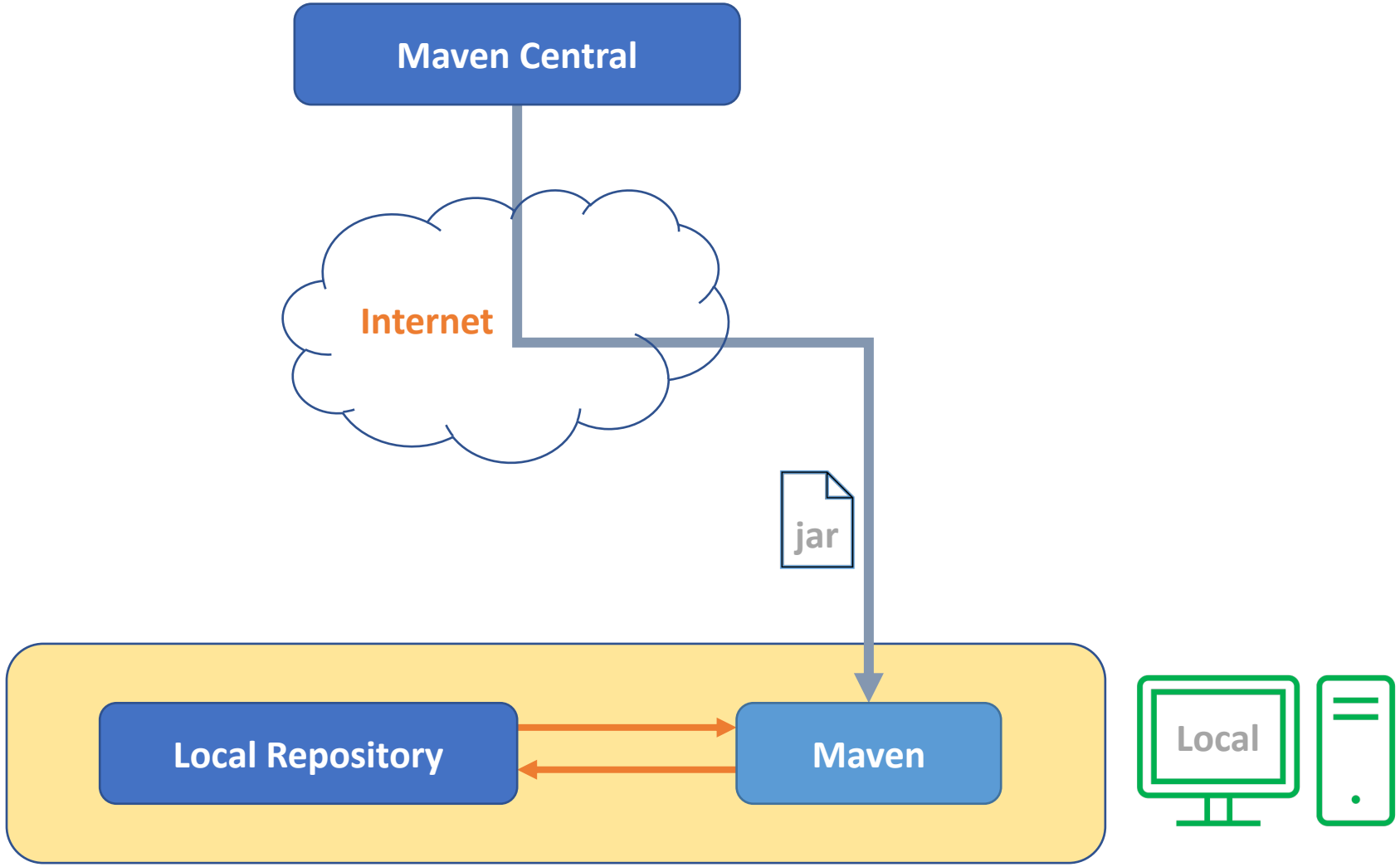
```
<dependency>
    <groupId>maven</groupId>
    <artifactId>project-a</artifactId>
    <version>1.0</version>
    <optional>true</optional>
</dependency>
<dependency>
    <groupId>maven</groupId>
    <artifactId>project-b</artifactId>
    <version>1.0</version>
    <optional>true</optional>
</dependency>
```

Example Use Case

- Suppose there is a project named ***X2*** that has similar functionality to Hibernate.
- It supports many databases such as MySQL, PostgreSQL, Oracle, ...
- Each supported database requires an additional dependency on a driver jar.
- All of these dependencies are needed at compile time to build X2.
- However, your project only uses one specific database
  and doesn't need drivers for the others.
- ***X2*** can declare these dependencies as optional, so that when your project declares ***X2*** as a direct dependency in its POM, all the drivers supported by the ***X2*** are not automatically included in your project's classpath.
- Your project will have to include an explicit dependency on the specific driver
  for the one database it does use.

Repositories

Default Maven Repository Architecture



Maven in Enterprise Environment

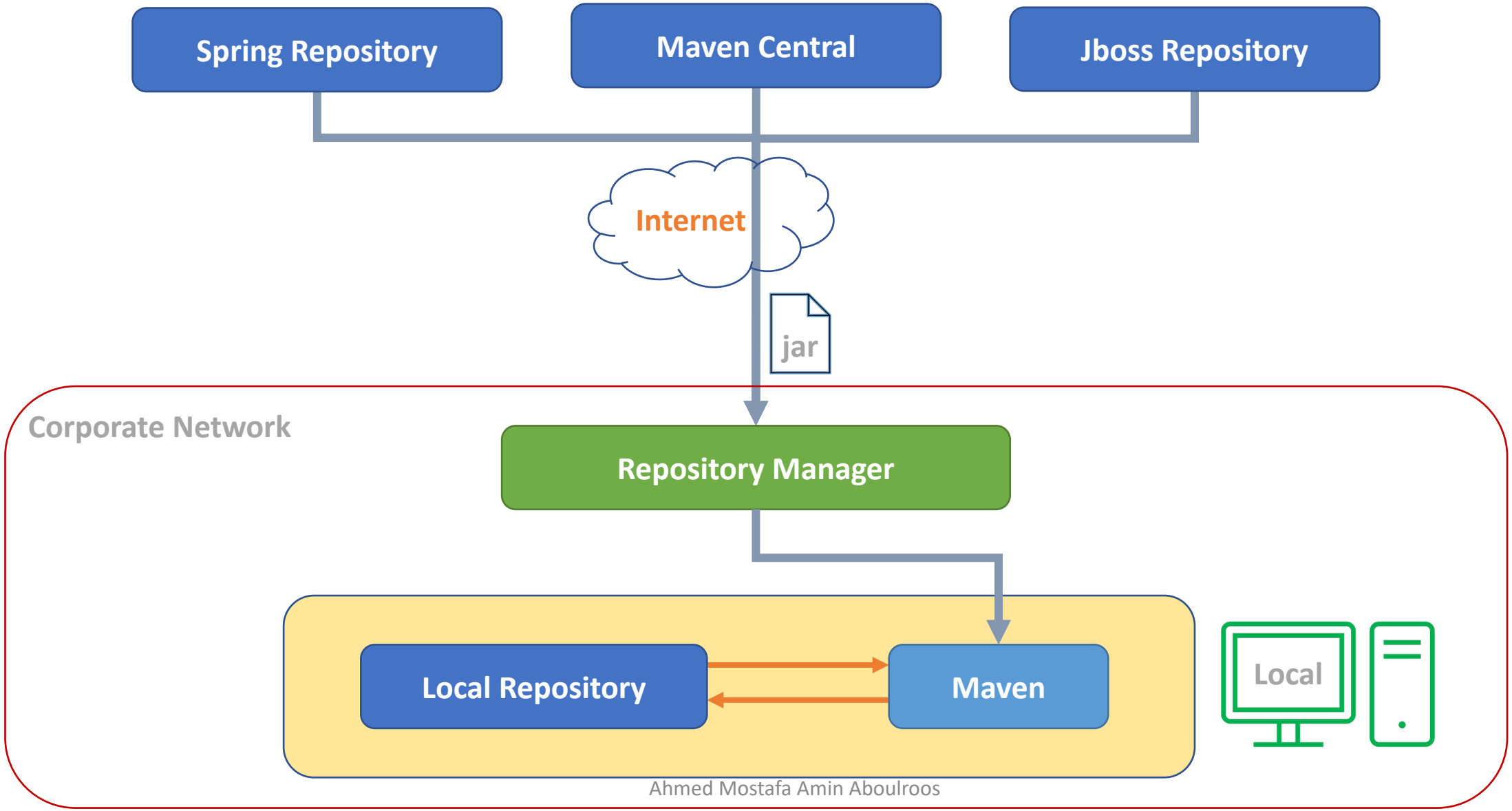
Although the architecture shown previously works in most cases,
it poses a few problems in an enterprise environment.

- The first problem is that ***sharing company-related artifacts*** between teams is not possible. Because of security and intellectual property concerns, you wouldn't want to publish your enterprise's artifacts on Maven Central.

- Another problem concerns ***legal and licensing issues***. Your company might want the teams only to use officially approved open-source software, and this architecture would not fit in that model.

- The final issue concerns ***bandwidth and download speeds***. In times of heavy load on Maven Central, the download speeds of Maven artifacts are reduced, and this might have a negative impact on your builds.

Enterprise Maven Repository Architecture



Enterprise Maven Repository Architecture

- The internal repository manager acts as a proxy to remote repositories.
- This allows you to cache artifacts from remote repositories resulting in faster artifact downloads and build performance improvements.
- Because you have full control over the internal repository, you can ***regulate*** the types of artifacts allowed in your company.
- Additionally, you can also push your organization's artifacts onto the repository manager, thereby enabling collaboration.
- There are several open-source repository managers
  - Nexus *(https://www.sonatype.com/products/repository-oss)*
  - Apache Archiva *(https://archiva.apache.org)*
  - Artifactory *(https://jfrog.com/artifactory)*

Maven Repositories

- A repository in Maven is used to hold build artifacts and dependencies of varying types.
- There are strictly only two types of repositories: **local** and **remote**.

- The ***local*** and ***remote*** repositories are ***structured the same way***,

  so that ***scripts can easily be run on either side***,

  or they can be ***synced for offline use***.

- Repository follows a simple directory structure
  - {groupId} / {artifactId} / {version} / {artifactId}-{version}.jar
  - And in the groupId '**.**' is replaced with '**/**'

Maven Local Repositories

- **Local Repository**
  - Refer to a copy on your own machine,
    that is a cache of the remote downloads.
  - Also contains the temporary build artifacts that you have not yet released.
  - Artifacts on which you have executed **CMD>** mvn install
  - Default Location in ***${user.home}/.m2/repository***

Maven Remote Repositories

- **Remote Repositories**
  - Refer to any other type of repository other than “Local Repository”.
  - Accessed by a variety of protocols such as ***file://*** and ***http://***.
  - These repositories might be a truly remote repository set up by a third party to provide their artifacts for downloading
  - Other "**remote**" repositories may be internal repositories set up on a file or HTTP server within your company, used to share private artifacts between development teams and for releases.

- In general use, the layout of the repositories is completely transparent to Maven users.
- The transportation of the dependencies is also transparent to the user, maven relies on Maven Wagon to download the dependencies, using http, ftp, file protocols internally.

Two Types of Repositories

- ***Dependencies Repository***
  - Holds artifacts of Dependencies
  - <repositories …>
- ***Plugins Repository***
  - Holds artifacts of plugins
  - <pluginRepositories …>

- Both (Dependencies and Plugins) can be stored on the same repository, but it’s recommended to split them in two different repositories.

- Both (Repository Types) have the same exact structure and rules, they only differ in what artifacts they hold/store either dependencies artifacts or plugins artifacts.

Plugins

Plugins

- Maven COC (*Conventions Over Configuration*) sets a lot of defaults,
  such as the standard directory structure,
  another COC defaults are the default built-in plugins that ship with maven.

- ***Maven is - at its heart - a plugin execution framework;***
  all work is done by plugins.

- One of the most powerful aspects of maven is its
  plugin-based architecture,
  everything is done through plugins.

Plugins

- In Maven, there are two kinds of plugins:
  - ***Build Plugins*** will be executed during the build
    and they should be configured in the *<build />* element from the POM.
  - ***Reporting Plugins*** will be executed during the site generation,
    and they should be configured in the *<reporting />* element from the POM.

- A plugin is a set of goals,
  - Either executed directly
    - **mvn** *<plugin-name>* **:** ***<goal-name>***
  - Or during a phase execution, if it's tied to a particular phase
    - **mvn** *<phase>*

Plugins

- Looking for a specific goal to execute?
- This page lists the core plugins and others.
  - https://maven.apache.org/plugins/index.html
  - ***Bookmark*** this web page

- Let's look at
  - ***Core plugins:*** clean, site, compiler, deploy, install, surefire
  - ***Packaging:*** jar, war, source, jlink, jmod
  - ***Reporting:*** Javadoc, jdeps, surefire-report
  - ***Tools:*** dependency, help, plugin

Getting Help

- Most plugins have a “help” goal, that describes the plugin and its available “goals” with a brief description of their function.

- CMD> **mvn** *<plugin-name>*:**help**
- CMD> **mvn** *<plugin-name>*:**help** **-D**detail=true
- CMD> **mvn** *<plugin-name>*:**help** **-D**goal=*<goal-name>*
- CMD> **mvn** *<plugin-name>*:**help** **-D**detail=true **-D**goal=*<goal-name>*

Plugin Prefix Resolution

- When you execute Maven using a standard lifecycle phase,
  resolving the plugins that participate in that lifecycle is a relatively simple process.

- However, when you directly invoke a mojo from the command line,
  as in the case of clean, Maven must have some way of reliably resolving the ***clean*** plugin prefix to the ***maven-clean-plugin***.

- This provides brevity for command-line invocations,
  while preserving the descriptiveness of the plugin's real artifactId.

Plugin Prefix Resolution

- By default, Maven will make a guess at the plugin-prefix to be used, by removing any instances of "maven" or "plugin" surrounded by hyphens in the plugin's artifact ID.

- The conventional artifact ID formats to use are:
  - ***maven-${prefix}-plugin***
    - For official plugins maintained by the Apache Maven team itself (you must not use this naming pattern for your plugin)
  - ***${prefix}-maven-plugin***
    - For plugins from other sources

Plugin Skeleton

```
<plugin>
    <groupId />
    <artifactId />
    <version />

    <executions />
    <configuration />
    <extensions />
    <dependencies />
    <goals />                    // Deprecated
    <inherited />
</plugin>
```

Plugin Executions

- When developing Plugins, the Goals can have a default phase binding.
  - If the goal has a default phase binding, then it will execute in that phase.
  - But if the goal is not bound to any lifecycle phase,
    then it simply won't be executed during the build lifecycle.
  - Specifying **Goal-to-Phase Binding**, Overrides the defaults.

```
<plugin>
    <groupId>org.codehaus.mojo</groupId>
    <artifactId>exec-maven-plugin</artifactId>
    <version>3.0.0</version>
    <executions>
        <execution>
            <phase>compile</phase>
            <goals>
                <goal>exec</goal>
            </goals>
        </execution>
    </executions>
</plugin>
```

Plugin Configurations

- Maven plugins (build and reporting) are configured by specifying a ***<configuration>*** element where the child elements of the <configuration> element are mapped to fields, or setters, inside your Mojo.
- (Remember that a plug-in consists of one or more Mojos where a Mojo maps to a goal.)
- Say, for example, you have a Mojo that performs a query against a particular URL, with a specified timeout and list of options.
- The Mojo might look like the following:

```
public class MyQueryMojo extends AbstractMojo {

    @Parameter(property = "query.url", required = true)
    private String url;

    @Parameter(property = "timeout", required = false, defaultValue = "50")
    private int timeout;

    @Parameter(property = "options")
    private String[] options;
}
```

Plugin Global Configurations

- To configure the Mojo from your POM with the desired URL, timeout and options you might have something like the following:

```
<project>
    ...
    <build>
        <plugins>
            <plugin>
                <artifactId>maven-myquery-plugin</artifactId>
                <version>1.0</version>
                <configuration>
                    <url>http://www.foobar.com/query</url>
                    <timeout>10</timeout>
                    <options>
                        <option>one</option>
                        <option>two</option>
                        <option>three</option>
                    </options>
                </configuration>
            </plugin>
        </plugins>
    </build>
    ...
</project>
```

Plugin Global Configurations

- For Mojos that are intended to be executed directly from the CLI,
  their parameters usually provide a means to be configured via system properties instead of a <configuration> section in the POM.
- The plugin documentation for those parameters will list an expression
  that denotes the system properties for the configuration.

- In the Mojo above, the parameter ***url*** is associated with the expression ***${query.url}***,
  meaning its value can be specified by the system property ***query.url*** as shown below:
  - CMD> **mvn** myquery**:**query **-D**query.url**=**http://maven.apache.org

- The name of the system property does not necessarily match the name of the mojo parameter. So be sure to have a close look at the plugin documentation.

Plugin Configurations per Execution

```
<plugin>
    <groupId>org.codehaus.mojo</groupId>
    <artifactId>exec-maven-plugin</artifactId>
    <version>3.0.0</version>

    <executions>
        <execution>
            <id>first-uid</id>
            <phase>validate</phase>
            <goals>
                <goal>java</goal>
            </goals>
            <configuration>
                <mainClass>amin.MainApp1</mainClass>
            </configuration>
        </execution>

        <execution>
            <id>second-uid</id>
            <phase>initialize</phase>
            <goals>
                <goal>java</goal>
            </goals>
            <configuration>
                <mainClass>amin.MainApp2</mainClass>
            </configuration>
        </execution>
    </executions>

</plugin>
```

The “help” Plugin

Maven Help Plugin

- The Maven Help Plugin provides goals aimed at
  helping to make sense out of the build environment.

- It includes the ability to view the effective POM and settings files,
  after inheritance and active profiles have been applied,
  as well as a describe a particular plugin goal to give usage information.

Help Plugin – Goals

Goal	Description
active-profiles	Displays a list of the profiles which are currently active for this build.
all-profiles	Displays a list of available profiles under the current project. Note: it will list all profiles for a project. If a profile comes up with a status inactive then there might be a need to set profile activation switches/property.
describe	Displays a list of the attributes for a Maven Plugin and/or goals (aka Mojo - Maven plain Old Java Object).
effective-pom	Displays the effective POM as an XML for this build, with the active profiles factored in, or a specified artifact. If verbose, a comment is added to each XML element describing the origin of the line.
effective-settings	Displays the calculated settings as XML for this project, given any profile enhancement and the inheritance of the global settings into the user-level settings.
evaluate	Evaluates Maven expressions given by the user in an interactive mode.
help	Display help information on maven-help-plugin. Call ***mvn help:help -Ddetail=true -Dgoal=<goal-name>*** to display parameter details.
system	Displays a list of the platform details like system properties and environment variables.

The *describe* Goal

- The ***describe*** goal is used to discover information about Maven plugins.

- If the user also specifies which goal to describe,
  the describe goal will limit output to the details of that goal,
  including parameters.

- You can execute this goal using the following command:
  - **mvn** help**:**describe
    -DgroupId**=**org.somewhere
    -DartifactId**=**some-plugin
    -Dversion**=**0.0.0

The *describe* Goal

- You must specify either:
  - both 'groupId' and 'artifactId' parameters
  - a 'plugin' parameter
  - a 'cmd' parameter.

- For instance:
  - **mvn** help**:**describe **-D**groupId**=**org.apache.maven.plugins **-D**artifactId**=**maven-help-plugin
  - **mvn** help**:**describe **-D**plugin**=**org.apache.maven.plugins**:**maven-help-plugin
  - **mvn** help**:**describe **-D**cmd**=**help:describe

- Option “**-D**detail**=**true” can be added to display extra information.
- Option “**-D**goal**=**evaluate” can be added to limit output to only that goal.

The *describe* Goal

- When you use the 'cmd' parameter, you can use it to describe a phase.
- For instance:
  - **mvn** help:describe **-D**cmd=clean
  - **mvn** help:describe **-D**cmd=site
  - **mvn** help:describe **-D**cmd=install
- And this will display the phase and which lifecycle it belongs to, and lists all the other phases in that lifecycle and their bindings.
- Additionally, it'll tell you about the packaging used if the phase is a default lifecycle phase.

The “compiler” Plugin

Maven & JDK9+

- By default, your version of Maven might use an old version of the ***maven-compiler-plugin*** that is not compatible with Java 9 or later versions.
- To target Java 9 or later, you should at least use version **3.6.0** of the ***maven-compiler-plugin*** and set the ***maven.compiler.release*** property to the Java release you are targeting (e.g., 9, 10, 11, 12, etc.).

- In the following example, we have configured our Maven project to use version **3.8.1** of ***maven-compiler-plugin*** and target ***Java 11***

Maven: *Java Project using Java 11*

```
<properties>
    <maven.compiler.release>11</maven.compiler.release>
</properties>


<build>
    <plugins>

        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-compiler-plugin</artifactId>
            <version>3.8.1</version>
        </plugin>

    </plugins>
</build>
```

Maven: *Java Project using Java 11*

```
<build>
    <plugins>

        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-compiler-plugin</artifactId>
            <version>3.8.1</version>
            <configuration>
                <release>11</release>
            </configuration>
        </plugin>

    </plugins>
</build>
```

The “exec” Plugin

Maven “exec” Plugin

- This is not an official plugin from maven, but it’s very handy.

```
<plugin>
    <groupId>org.codehaus.mojo</groupId>
    <artifactId>exec-maven-plugin</artifactId>
    <version>3.0.0</version>
</plugin>
```

- CMD> **mvn** exex:help **-D**detail

Goal	Description
exec	A Goal for executing external programs.
java	Executes the supplied java class in the current VM with the enclosing project's dependencies as classpath.
help	Display help information on exec-maven-plugin. Call mvn exec:help -Ddetail=true -Dgoal=<goal-name> to display parameter details.

The *java* Goal

- The ***java*** goal is used to execute the supplied java class in the current VM with the enclosing project's dependencies as classpath.

```
<plugin>
    <groupId>org.codehaus.mojo</groupId>
    <artifactId>exec-maven-plugin</artifactId>
    <version>3.0.0</version>
    <configuration>
        <mainClass>com.example.MainApp</mainClass>
    </configuration>
</plugin>
```

- CMD> mvn exec:java
  - NOTE: You must compile the classes before running the application.

Specifying Main-Class from Terminal

- You can pass the parameters used in the configuration section at runtime instead of writing them in the pom.xml, which is very useful in case that those configurations change very often.

- In the pom.xml

```
<plugin>
    <groupId>org.codehaus.mojo</groupId>
    <artifactId>exec-maven-plugin</artifactId>
    <version>3.0.0</version>
    <!-- No Configurations Specified -->
</plugin>
```

- CMD> **mvn** exec:java **-D**exec.mainClass=com.example.MainApp

Binding to a Phase

- This way every call to ***mvn compile*** will also run the application.

```
<plugin>
    <groupId>org.codehaus.mojo</groupId>
    <artifactId>exec-maven-plugin</artifactId>
    <version>3.0.0</version>
    <executions>
        <execution>
            <phase>compile</phase>
            <goals>
                <goal>java</goal>
            </goals>
        </execution>
    </executions>
    <configuration>
        <mainClass>com.example.MainApp</mainClass>
    </configuration>
</plugin>
```

The *exec* Goal

- The ***exec*** goal is used to execute an external program.

```
<plugin>
    <groupId>org.codehaus.mojo</groupId>
    <artifactId>exec-maven-plugin</artifactId>
    <version>3.0.0</version>
    <executions>
        <execution>
            <phase>compile</phase>
            <goals>
                <goal>exec</goal>
            </goals>
        </execution>
    </executions>
    <configuration>
        <executable>notepad</executable>
        <arguments>
            <argument>pom.xml</argument>
        </arguments>
    </configuration>
</plugin>
```

Now whenever you run
CMD> mvn compile

The plugin will execute
the external program “notepad”
With “pom.xml” as argument.

Project Inheritance

Project Inheritance

- Maven supports Project Inheritance,
  Where a Project can inherit from a single Parent Project,
  This results in Overriding and Merging of elements in POM files.

- Elements in the POM that are merged are the following:
  - dependencies
  - developers and contributors
  - plugin lists (including reports)
  - plugin executions with matching ids
  - plugin configuration
  - resources

- You can introduce parent POMs by specifying the parent element in the project POM, as demonstrated in the following examples.

Example 1

```
<project>
    <modelVersion> 4.0.0 </modelVersion>
    <groupId> com.example </groupId>
    <artifactId> my-parent </artifactId>
    <version> 1 </version>
</project>
```

```
<project>
    <modelVersion> 4.0.0 </modelVersion>
    <groupId> com.example </groupId>
    <artifactId> my-project </artifactId>
    <version> 1 </version>
</project>
```

```
-- my-parent
|
|-- my-project
|    `-- pom.xml
|
 `-- pom.xml
```

Now, if we were to turn
com.example:my-parent:1
into a parent artifact of
com.example:my-project:1
we will have to modify my-parent and my-project POMs as following:

Example 1, Solution

```
<project>

        <modelVersion> 4.0.0 </modelVersion>
        <groupId> com.example </groupId>
        <artifactId> my-parent </artifactId>
        <version> 1 </version>
        <packaging> pom </packaging>

</project>
```

The parent project “my-parent” must be of type pom, so that it can be inherited.

Example 1, Solution

```
<project>
   <modelVersion> 4.0.0 </modelVersion>
   <groupId> com.example </groupId>
   <artifactId> my-project </artifactId>
   <version> 1 </version>
   <parent>
     <groupId> com.example </groupId>
     <artifactId> my-parent </artifactId>
     <version> 1 </version>
   </parent>
</project>
```

This Parent section allows us to specify which artifact is the parent of our POM. And we do so by specifying the fully qualified artifact name of the parent POM. With this setup, our module can now inherit some of the properties of our parent POM.

```
<project>
   <modelVersion> 4.0.0 </modelVersion>

   <artifactId> my-project </artifactId>

   <parent>
     <groupId> com.example </groupId>
     <artifactId> my-parent </artifactId>
     <version> 1 </version>
   </parent>
</project>
```

Alternatively, if we want the groupId and / or the version of your modules to be the same as their parents, you can remove the groupId and / or the version identity of your module in its POM.

Example 2, Solution

However, that would work if the parent project was already installed in our local repository
or was in that specific directory structure
(parent pom.xml is one directory higher than that of the module's pom.xml).

But what if the parent is not yet installed and if the directory structure is as in the following example?

```
-- projects
|
|-- my-project
|    `-- pom.xml
|
|-- my-parent
|    `-- pom.xml
|
.
```

```
<project>
    <modelVersion> 4.0.0 </modelVersion>
    <artifactId> my-project </artifactId>
    <parent>
        <groupId> com.example </groupId>
        <artifactId> my-parent </artifactId>
        <version> 1 </version>
        <relativePath>../my-parent/pom.xml</relativePath>
    </parent>
</project>
```

Example 3, Solution

What about Parents that are not on your local machine ?
e.g., Frameworks such as “Spring Boot” publish a Parent Project on Maven Central for their users.

```
-- my-project
 `-- pom.xml
```

- You could declare an empty relativePath element.
In which case maven will attempt to download the parent pom from the repository directly.

```
<project>
    <modelVersion> 4.0.0 </modelVersion>
    <artifactId> my-project </artifactId>
    <parent>
        <groupId> com.example </groupId>
        <artifactId> some-online-parent </artifactId>
        <version> 2.3.0-FINAL </version>
        <relativePath />
    </parent>
</project>
```

- Also, You could omit the relativePath element altogether.
In which case maven by default will look for the parent pom in the parent folder of the child.
When it’s not found it’ll attempt to download the parent pom from the repository.

Project Aggregation

Project Aggregation

- Project Aggregation is when a project declares other projects as its modules.
- By doing so, if a Maven command is invoked against the aggregate project, that Maven command will then be executed to its modules as well.
- To do Project Aggregation, you must do the following:
  - Change the aggregate project’s packaging to "pom".
  - Specify in the aggregate project’s POM file the directories of its modules.

Example 1

```
<project>
    <modelVersion> 4.0.0 </modelVersion>
    <groupId> com.example </groupId>
    <artifactId> my-aggregate </artifactId>
    <version> 1 </version>
</project>
```

```
<project>
    <modelVersion> 4.0.0 </modelVersion>
    <groupId> com.example </groupId>
    <artifactId> my-first-module </artifactId>
    <version> 1 </version>
</project>
```

```
 -- my-aggregate
 |
 |-- my-first-module
 |    `-- pom.xml
 |
 |-- my-second-module
 |    `-- pom.xml
 |
 |-- my-third-module
 |    `-- pom.xml
 |
  `-- pom.xml
```

```
<project>
    <modelVersion> 4.0.0 </modelVersion>
    <groupId> com.example </groupId>
    <artifactId> my-second-module </artifactId>
    <version> 1 </version>
</project>
```

```
<project>
    <modelVersion> 4.0.0 </modelVersion>
    <groupId> com.example </groupId>
    <artifactId> my-third-module </artifactId>
    <version> 1 </version>
</project>
```

Example 1, Solution

```
<project>

    <modelVersion>4.0.0</modelVersion>

    <groupId> com.example </groupId>
    <artifactId> my-aggregate </artifactId>
    <version> 1 </version>

    <packaging> pom </packaging>

    <modules>
        <module> my-first-module </module>
        <module> my-second-module </module>
        <module> my-third-module </module>
    </modules>

</project>
```

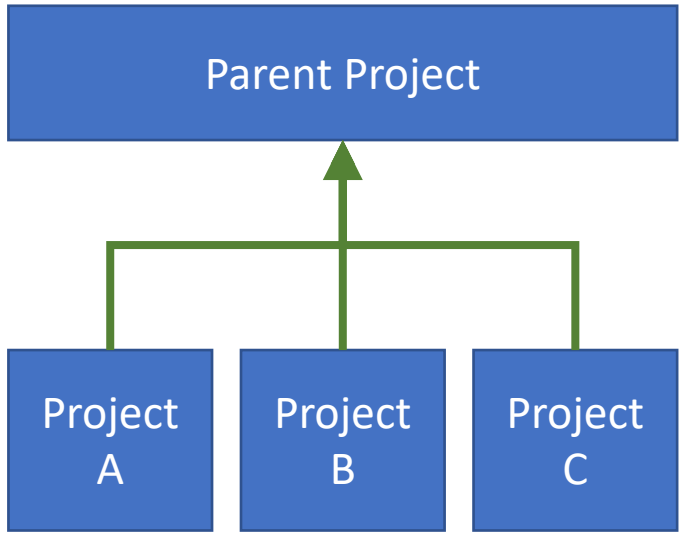
For the ***packaging***, its value was set to "***pom***", and for the modules section, we have the elements
<module> my-xxx-module </module>

The value of ***<module>*** is the ***relative path***
(by practice, we use the module's artifactId
as the module directory's name).

Now, whenever a Maven command issued against
com.example : my-aggregate : 1
that same Maven command would run against
com.example : my-first-module : 1
com.example : my-second-module : 1
com.example : my-third-module : 1
as well.

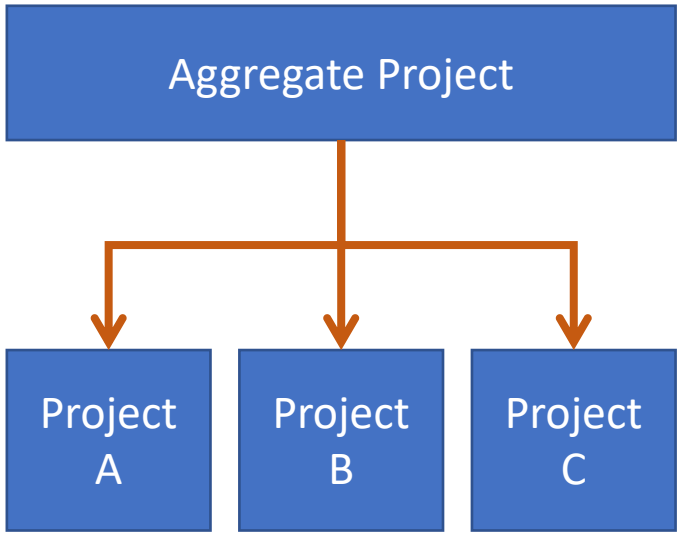
Multi-Module Projects

Project Inheritance **vs** Project Aggregation



- Parent Doesn't Know about its Childs.
- Child Knows about it's Parent.

Why use Inheritance ?
- to group and share commonalities among several projects.



- Parent Knows about its Childs.
- Child Doesn't Know about it's Parent.

Why use Aggregation ?
- to invoke the same Maven Command over multiple related projects.

Project Inheritance **vs** Project Aggregation

- If you have several Maven projects, and they all have similar configurations, you can refactor your projects by pulling out those similar configurations and making a parent project.
- Thus, all you have to do is to let your Maven projects inherit that parent project, and those configurations would then be applied to all of them.

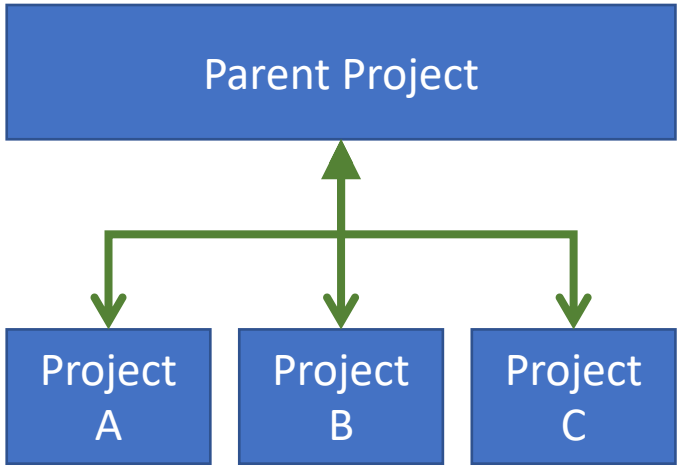
- And if you have a group of projects that are built or processed together, you can create an aggregate project and have that aggregate project declare those projects as its modules.
- By doing so, you'd only have to build the aggregate and the rest will follow.

Project Inheritance **vs** Project Aggregation

- But of course, you can have both Project Inheritance and Project Aggregation.

- Meaning, you can have your modules specify a parent project, and at the same time, have that parent project specify those Maven projects as its modules.

- You'd just have to apply all three rules:
  - Change the parent POMs packaging to the value "pom"
  - Specify in the parent POM the directories of its modules
  - Specify in every child POM who their parent POM is



Multimodule Project

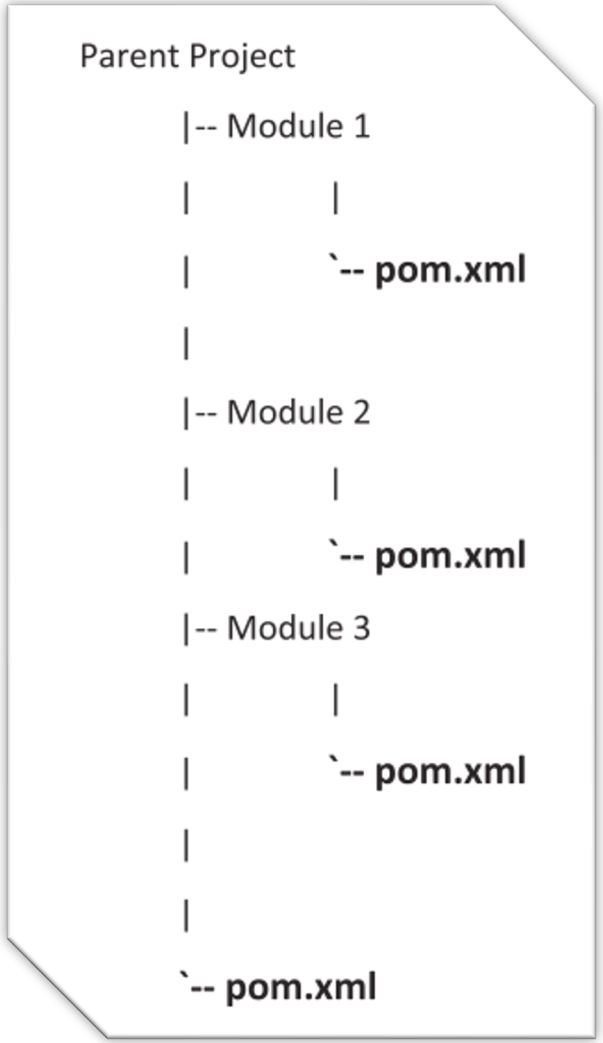
- Java Enterprise projects are often split into several modules to ease development and maintainability.

- Each of these modules produces artifacts such as Enterprise JavaBeans (EJBs), web services, web projects, and client jars.

- Maven supports development of such large projects by allowing multiple Maven projects to be nested under a single Maven project.

Multimodule Project Structure

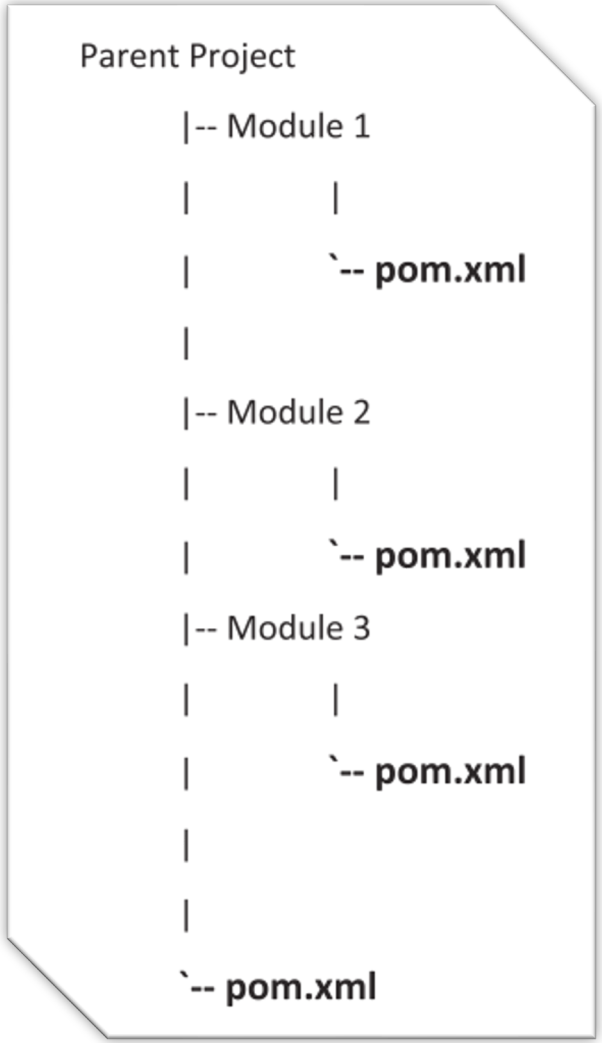
- This is the layout of such a multimodule project.
- The parent project has a pom.xml file and individual Maven projects inside it.



Multimodule Project Structure

- Parent Project > pom.xml

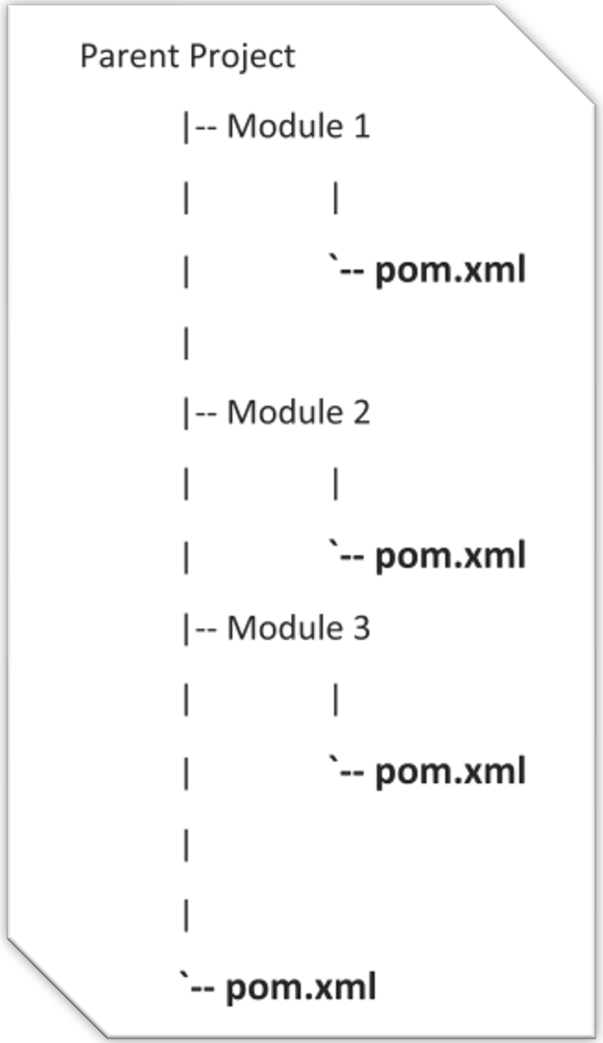
```
<project xmlns="…" xmlns:xsi="…" xsi:schemaLocation="…">
    <modelVersion> 4.0.0 </modelVersion>
    <groupId> com.example </groupId>
    <artifactId> parent-project </artifactId>
    <version> 1.0.0-SNAPSHOT </version>
    <packaging> pom </packaging>
    <modules>
        <module> module-1 </module>
        <module> module-2 </module>
        <module> module-3 </module>
    </modules>
</project>
```



Multimodule Project Structure

- Child Module > pom.xml

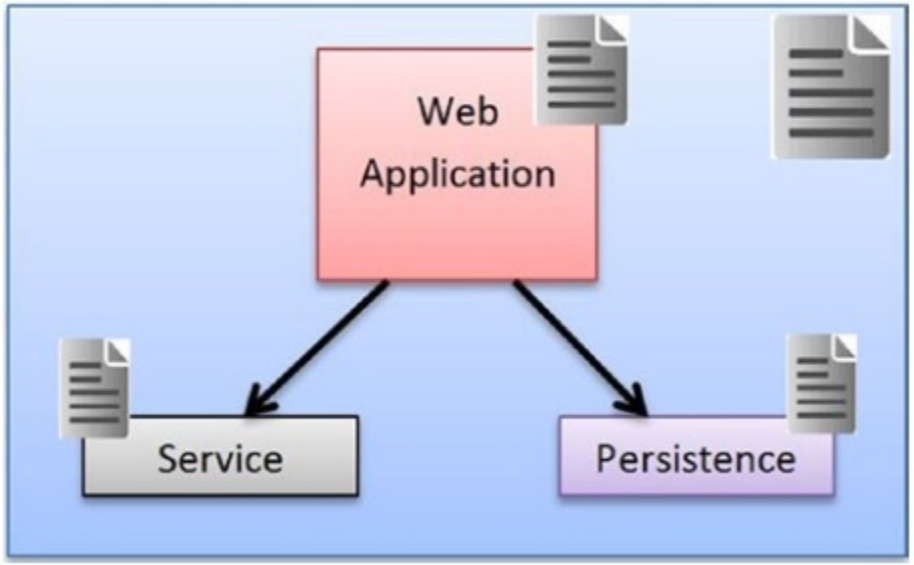
```
<project xmlns="…" xmlns:xsi="…" xsi:schemaLocation="…">
    <modelVersion> 4.0.0 </modelVersion>
    <groupId> com.example </groupId>
    <artifactId> module-1 </artifactId>
    <version> 1.0.0-SNAPSHOT </version>
    <packaging> jar </packaging>
    <parent>
        <groupId> com.example </groupId>
        <artifactId> parent-project </artifactId>
        <version> 1.0.0-SNAPSHOT </version>
    </parent>
</project>
```



Example

- Assume we are starting out a new web-based project, with the following layered architecture
  - Web layer
  - Service layer
  - Persistence layer

- What maven structure would we need to make to achieve separation between these layers?



Generating the Parent Project

- Let’s start the process by generating the parent project.

```
mvn archetype:generate
        -DgroupId=com.example
        -DartifactId=parent-project
        -Dversion=1.0.0-SNAPSHOT
        -DarchetypeGroupId=org.codehaus.mojo.archetypes
        -DarchetypeArtifactId=pom-root
```

Creating the Web Module

- Generate the web project/module inside the parent project.

```
mvn archetype:generate
        -DgroupId=com.example
        -DartifactId=my-web
        -Dversion=1.0.0-SNAPSHOT
        -Dpackage=war
        -DarchetypeArtifactId=maven-archetype-webapp
```

Creating the Service Module

- Generate the service project/module inside the parent project.

```
mvn archetype:generate
        -DgroupId=com.example
        -DartifactId=my-service
        -Dversion=1.0.0-SNAPSHOT
        -DarchetypeArtifactId=maven-archetype-quickstart
        -DinteractiveMode=false
```

Creating the Persistence Module

- Generate the persistence project/module inside the parent project.

```
mvn archetype:generate
        -DgroupId=com.example
        -DartifactId=my-persistence
        -Dversion=1.0.0-SNAPSHOT
        -DarchetypeArtifactId=maven-archetype-quickstart
        -DinteractiveMode=false
```

DEMO

- First, we generated the parent pom project.
- Then we generated 3 projects/modules inside the parent project.

- As you generated each module,
  Maven intelligently registered them as a child module.

- Additionally, it modified the individual module's pom.xml file
  and added the parent pom information.

Dependency Management

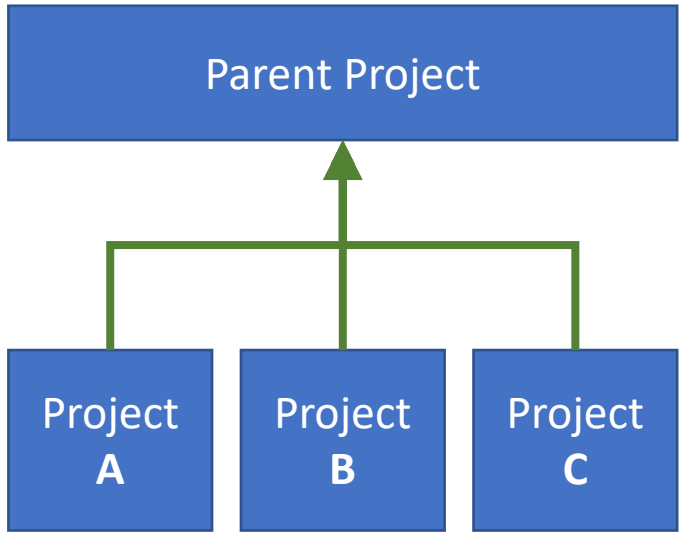
Dependency Management

- Dependency Management
  - A mechanism to centralize dependency information.

- When there are a set of projects (or modules) that inherit a common parent, all the information about the dependency can be put in the parent POM and the projects can have simpler references to them.

- This makes it easy to maintain the dependencies across multiple projects and reduces the issues that typically arise due to multiple versions of the same dependencies.

Dependency Management




```
<dependencies>
    <dependency>
        <groupId>org.junit.jupiter</groupId>
        <artifactId>junit-jupiter-api</artifactId>
        <version>5.7.2</version>
        <scope>test</scope>
    </dependency>
</dependencies>
```

Now all these Projects A & B & C
Have that “junit” Dependency.

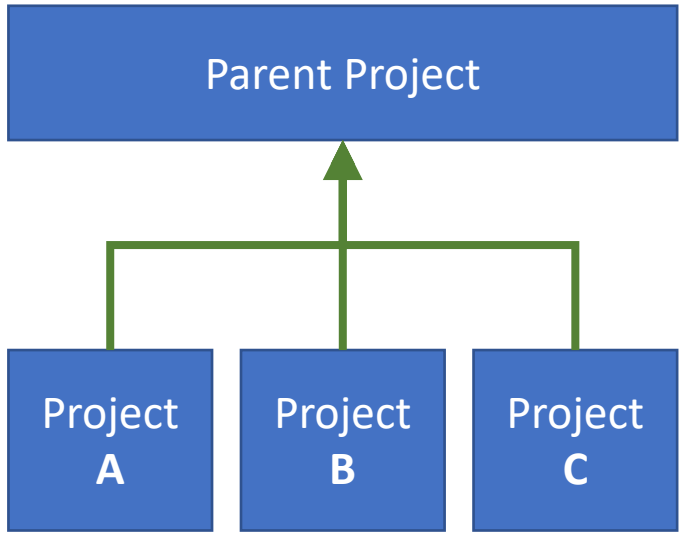
By the virtue of inheritance.

Q. What if it’s Only needed in Projects A & B.

Q. If it’s declared in both A & B,
- aren’t you repeating yourself ?
- what happens when updating and forgetting one ?
- is it possible to have different versions of the same dependency shipped with your application ?

Q. Can you ensure that the newly added dependencies in the projects A & B & C will be compatible with those inherited from the parent ?

Dependency Management




```xml
<dependencyManagement>
    <dependencies>
        <dependency>
            <groupId>org.junit.jupiter</groupId>
            <artifactId>junit-jupiter-api</artifactId>
            <version>5.7.2</version>
            <scope>test</scope>
        </dependency>
    </dependencies>
</dependencyManagement>
```

None of the Child Projects have the “junit” Dependency unless explicitly specified.

Specifying it as a dependency, doesn’t require version nor scope information.

```xml
<dependencies>
    <dependency>
        <groupId>org.junit.jupiter</groupId>
        <artifactId>junit-jupiter-api</artifactId>
    </dependency>
</dependencies>
```

How it works

- Dependencies that are specified within the “dependencyManagement” section of the parent POM are available for use by all the child projects.
- The child project needs to choose the dependencies by explicitly specifying the required dependencies in the “dependencies” section.
- While doing this the child can omit the “version” and “scope” information so that they are inherited from the parent.

- Why not put all the dependencies in the “dependencies” section ?
  - A child project typically needs some of the dependencies that the parent defines.
  - The “dependencyManagement” section
    - Allows child projects to selectively choose these not all of them.
    - Helps address any surprises of Maven’s Dependency Mediation.

Plugin Management

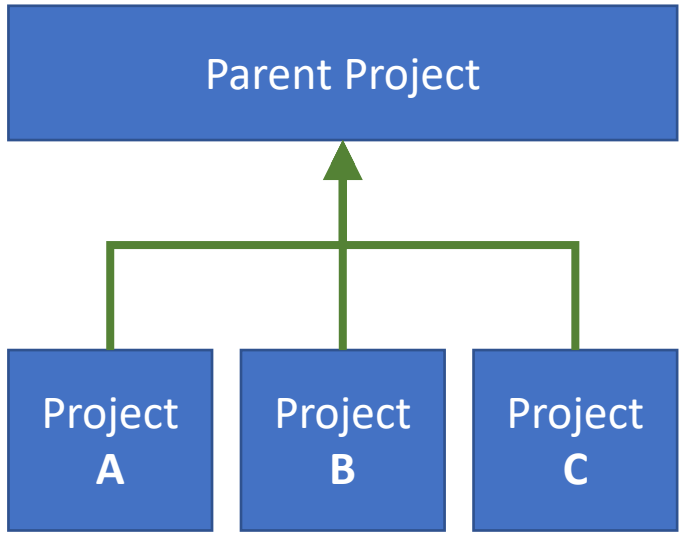
Plugin Management

- Plugin Management
  - A mechanism to configure plugin information that can be used as required by child projects.

- The parent POM can define the configurations for various plugins used by different child projects.

- Each child project can choose the plugins that it needs for the build.

Plugin Management




```
<build>
   <plugins>
      <plugin>
         <groupId>org.codehaus.mojo</groupId>
         <artifactId>exec-maven-plugin</artifactId>
         <version>3.0.0</version>
      </plugin>
   </plugins>
</build>
```

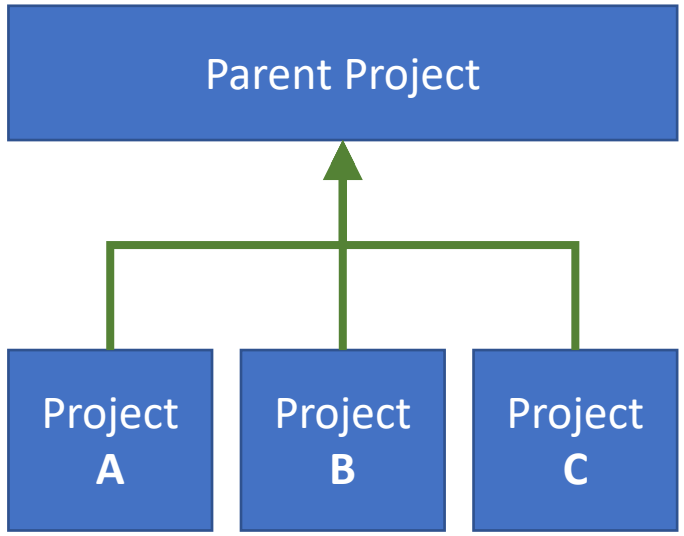
Now all these Projects A & B & C
Have that “exec” Plugin.

By the virtue of inheritance.

Q. What if it’s Only needed in Projects A & B.

Q. If it’s declared in both A & B,
- aren’t you repeating yourself ?
- what happens when updating and forgetting one ?
- is it possible to have different versions of the same plugin used within your application ?

Plugin Management




```xml
<pluginManagement>
    <plugins>
        <plugin>
            <groupId>org.codehaus.mojo</groupId>
            <artifactId>exec-maven-plugin</artifactId>
            <version>3.0.0</version>
            <executions>
                <execution>
                    <phase>package</phase>
                    <goals> <goal>java</goal> </goals>
                </execution>
            </executions>
        </plugin>
    </plugins>
</pluginManagement>
```

None of the Child Projects have the "exec" Plugin unless explicitly specified.

Specifying it as a plugin, brings all the Information configured for the plugin In the parent project, and you can add, merge and override them.

```xml
<plugins>
    <plugin>
        <groupId>org.codehaus.mojo</groupId>
        <artifactId>exec-maven-plugin</artifactId>
        <configuration>
            <mainClass>io.amin.MyMainApp</mainClass>
        </configuration>
    </plugin>
</plugins>
```

One More Thing

- ***If a plugin is used as part of the build lifecycle, then its configurations in the "pluginManagement" section will take effect, even if not explicitly defined by the child.***
  - For Example, the "compile" goal of the "compiler" plugin is used in the "build" lifecycle.
  - It will take effect in the child project, even if the child project didn't explicitly define it.

```
<pluginManagement>
    <plugins>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-compiler-plugin</artifactId>
            <version>3.8.1</version>
            <configuration>
                <release>11</release>
            </configuration>
        </plugin>
    </plugins>
</pluginManagement>
```

- To disable plugin information inheritance set the "inherited" element value in the plugin to false.

Plugin Inheritance

- By default, ***plugin configuration should be propagated to child POMs***, so, ***to break the inheritance***, you could use the ***<inherited>*** tag:

```
<project>
    ...
    <build>
      <plugins>
        <plugin>
          <groupId>org.apache.maven.plugins</groupId>
          <artifactId>maven-antrun-plugin</artifactId>
          <version>1.2</version>
          <inherited>false</inherited>
          ...
        </plugin>
      </plugins>
    </build>
    ...
</project>
```

Importing Dependencies

Importing Dependencies

- The examples in the previous section describe
  How to specify managed dependencies through ***inheritance***.

- However, in larger projects it may be impossible to accomplish this
  since ***a project can only inherit from a single parent***.

- To accommodate this,
  projects can ***import managed dependencies*** from other projects.

- ***This is accomplished by***
  declaring a POM artifact as a dependency with a scope of "import".

Importing Dependencies

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/maven-v4_0_0.xsd">

  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>my-dependencies</artifactId>
  <packaging>pom</packaging>
  <version>1.0</version>

  <dependencyManagement>
    <dependencies>

      <dependency>
        <groupId>jakarta.xml.bind</groupId>
        <artifactId>jakarta.xml.bind-api</artifactId>
        <version>3.0.1</version>
      </dependency>

    </dependencies>
  </dependencyManagement>

</project>
```

Importing Dependencies

```
<project ... >

    <groupId>com.example</groupId>
    <artifactId>my-project</artifactId>
    <version>1.0-SNAPSHOT</version>

    <dependencyManagement>
       <dependencies>

          <dependency>
              <groupId>com.example</groupId>
              <artifactId>my-dependencies</artifactId>
              <version>1.0</version>
              <type>pom</type>
              <scope>import</scope>
          </dependency>

       </dependencies>
    </dependencyManagement>

    <dependencies>
        <dependency>
            <groupId>jakarta.xml.bind</groupId>
            <artifactId>jakarta.xml.bind-api</artifactId>
        </dependency>
    </dependencies>

</project>
```

Bill of Materials (BOM) POMs

Bill of Materials (BOM) POMs

- Imports are most effective when used for defining a "library" of related artifacts that are generally part of a multi-project build.

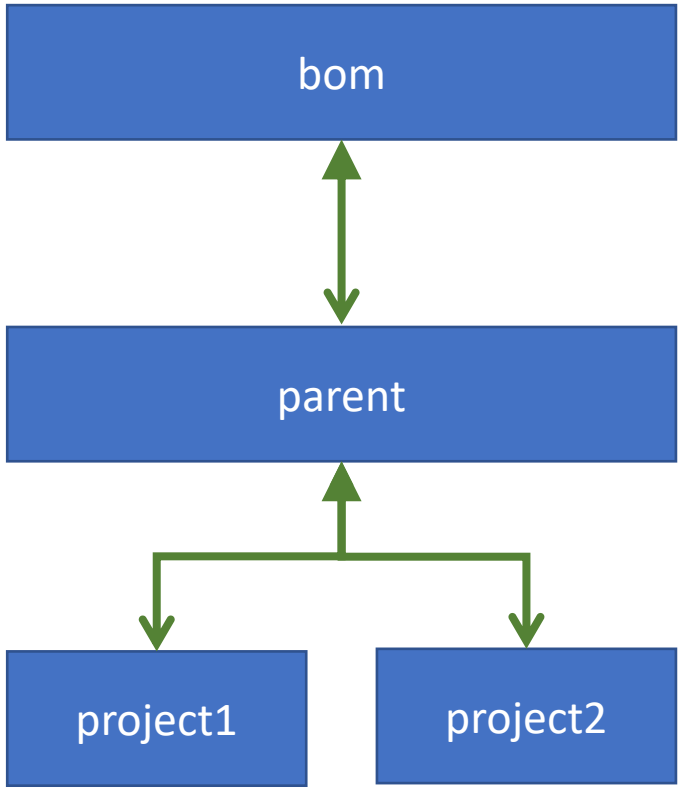
- It is fairly common for one project to use one or more artifacts from these libraries.

- However, it has sometimes been difficult to keep the versions in the project using the artifacts in sync with the versions distributed in the library.

- The pattern below illustrates how a "bill of materials" (BOM) can be created for use by other projects.

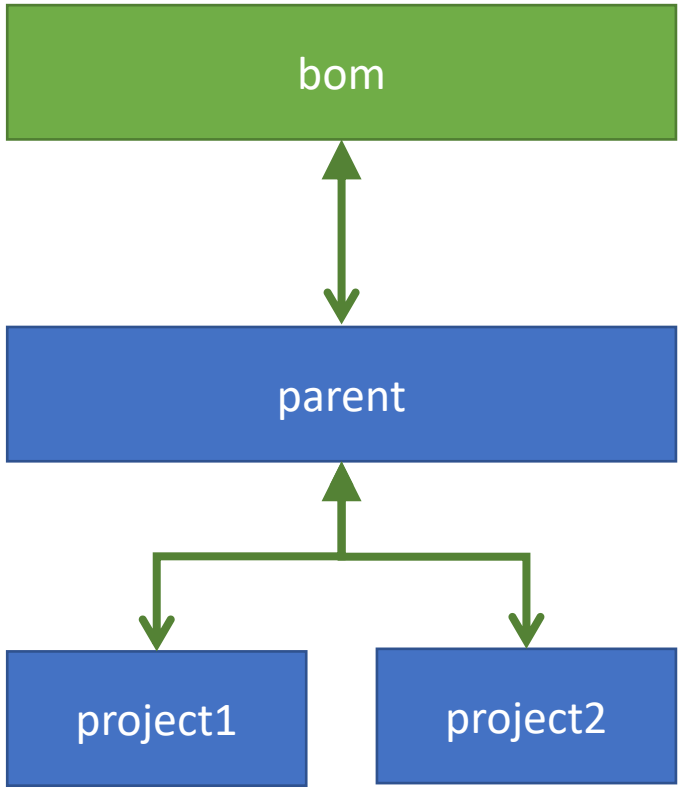
Bill of Materials (BOM) POMs

- The root of the project is the BOM POM.
- It defines the versions of all the artifacts that will be created in the library.
- Other projects that wish to use the library should import this POM into the dependencyManagement section of their POM.
- The parent subproject has the BOM POM as its parent. It is a normal multi-project pom.



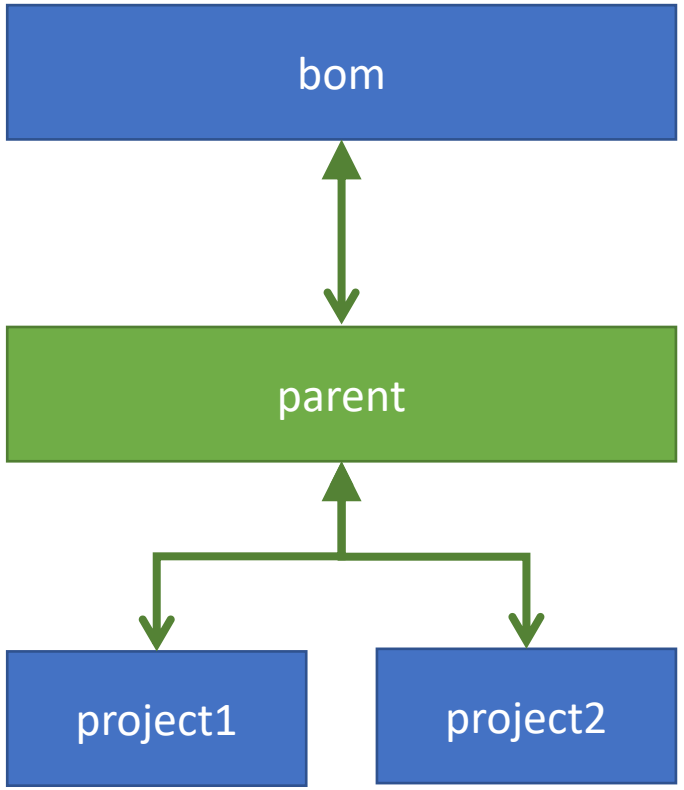
Bill of Materials (BOM) POMs

```
<project ... >
    <groupId>com.test</groupId>
    <artifactId>bom</artifactId>
    <version>1.0.0</version>
    <packaging>pom</packaging>

    <properties>
        <project1Version>1.0.0</project1Version>
        <project2Version>1.0.0</project2Version>
    </properties>

    <dependencyManagement>
        <dependencies>
            <dependency>
                <groupId>com.test</groupId>
                <artifactId>project1</artifactId>
                <version>${project1Version}</version>
            </dependency>
            <dependency>
                <groupId>com.test</groupId>
                <artifactId>project2</artifactId>
                <version>${project2Version}</version>
            </dependency>
        </dependencies>
    </dependencyManagement>

    <modules>
        <module>parent</module>
    </modules>

</project>
```



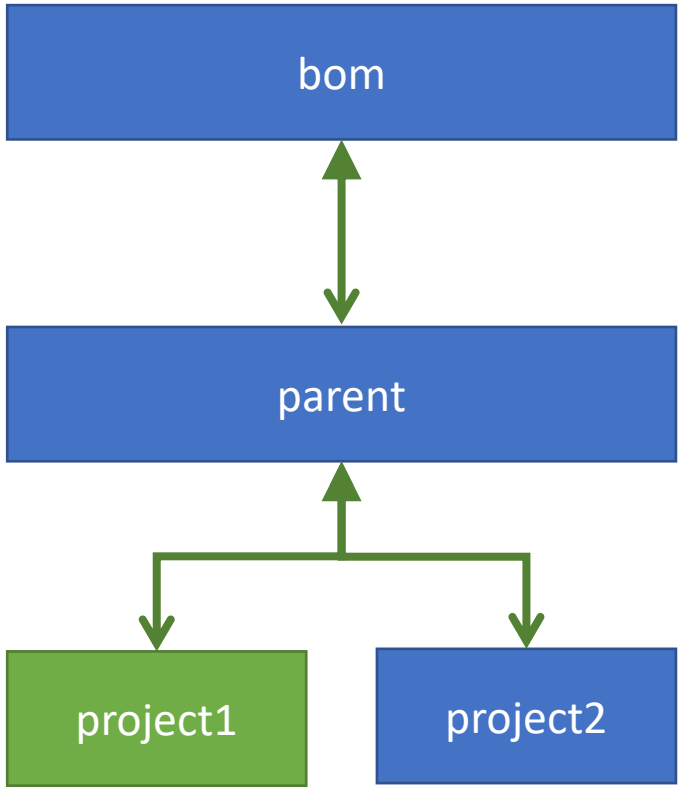
Bill of Materials (BOM) POMs

```
<project ... >
    <parent>
        <groupId>com.test</groupId>
        <version>1.0.0</version>
        <artifactId>bom</artifactId>
    </parent>

    <groupId>com.test</groupId>
    <artifactId>parent</artifactId>
    <version>1.0.0</version>
    <packaging>pom</packaging>

    <dependencyManagement>
        <dependencies>
            <dependency>
                <groupId>log4j</groupId>
                <artifactId>log4j</artifactId>
                <version>1.2.12</version>
            </dependency>
            <dependency>
                <groupId>commons-logging</groupId>
                <artifactId>commons-logging</artifactId>
                <version>1.1.1</version>
            </dependency>
        </dependencies>
    </dependencyManagement>

    <modules>
        <module>project1</module>
        <module>project2</module>
    </modules>
</project>
```



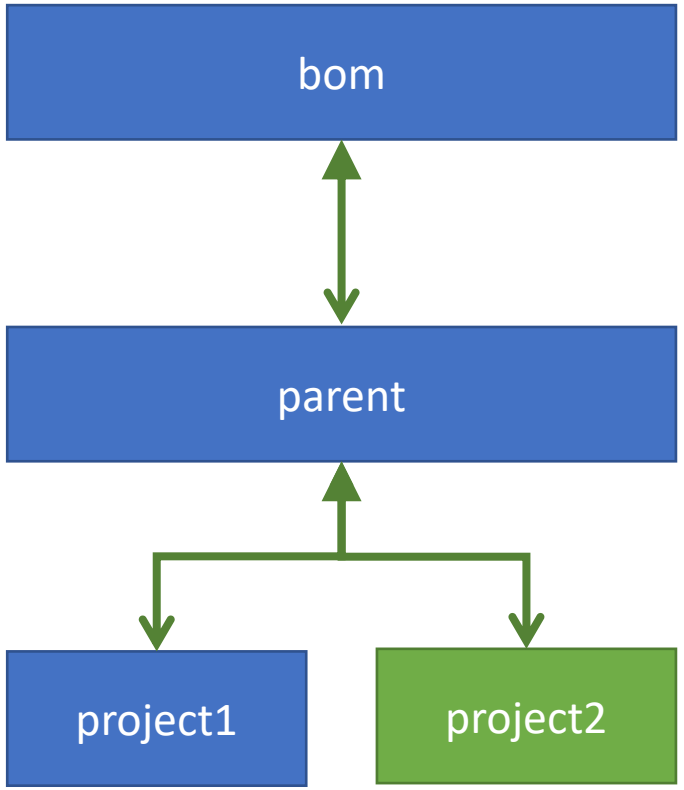
Bill of Materials (BOM) POMs

```
<project ... >

    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>com.test</groupId>
        <version>1.0.0</version>
        <artifactId>parent</artifactId>
    </parent>

    <groupId>com.test</groupId>
    <artifactId>project1</artifactId>
    <version>${project1Version}</version>
    <packaging>jar</packaging>

    <dependencies>
        <dependency>
            <groupId>log4j</groupId>
            <artifactId>log4j</artifactId>
        </dependency>
    </dependencies>

</project>
```



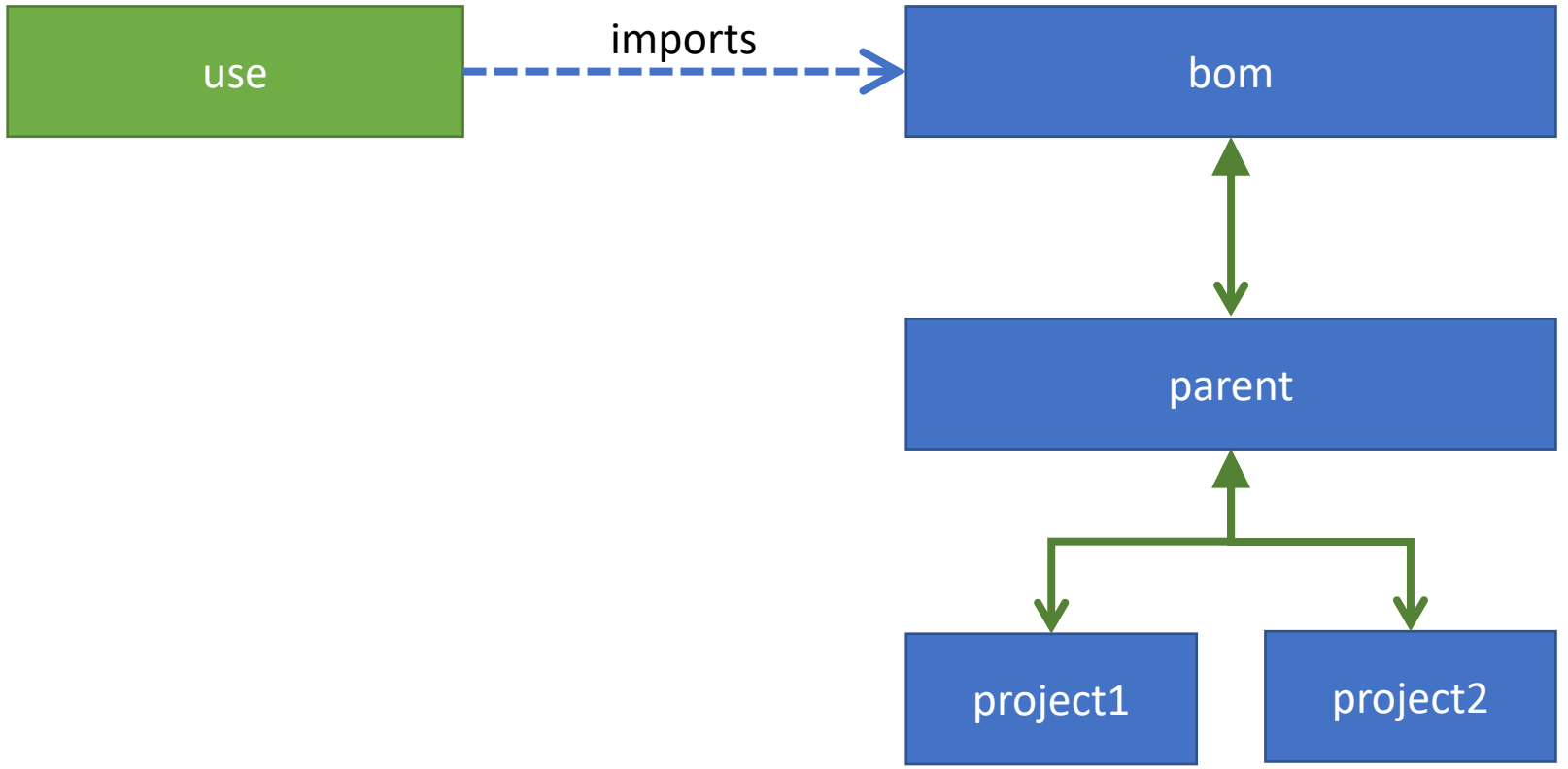
Bill of Materials (BOM) POMs

```
<project ... >

    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>com.test</groupId>
        <version>1.0.0</version>
        <artifactId>parent</artifactId>
    </parent>

    <groupId>com.test</groupId>
    <artifactId>project2</artifactId>
    <version>${project2Version}</version>
    <packaging>jar</packaging>

    <dependencies>
        <dependency>
            <groupId>commons-logging</groupId>
            <artifactId>commons-logging</artifactId>
        </dependency>
    </dependencies>

</project>
```



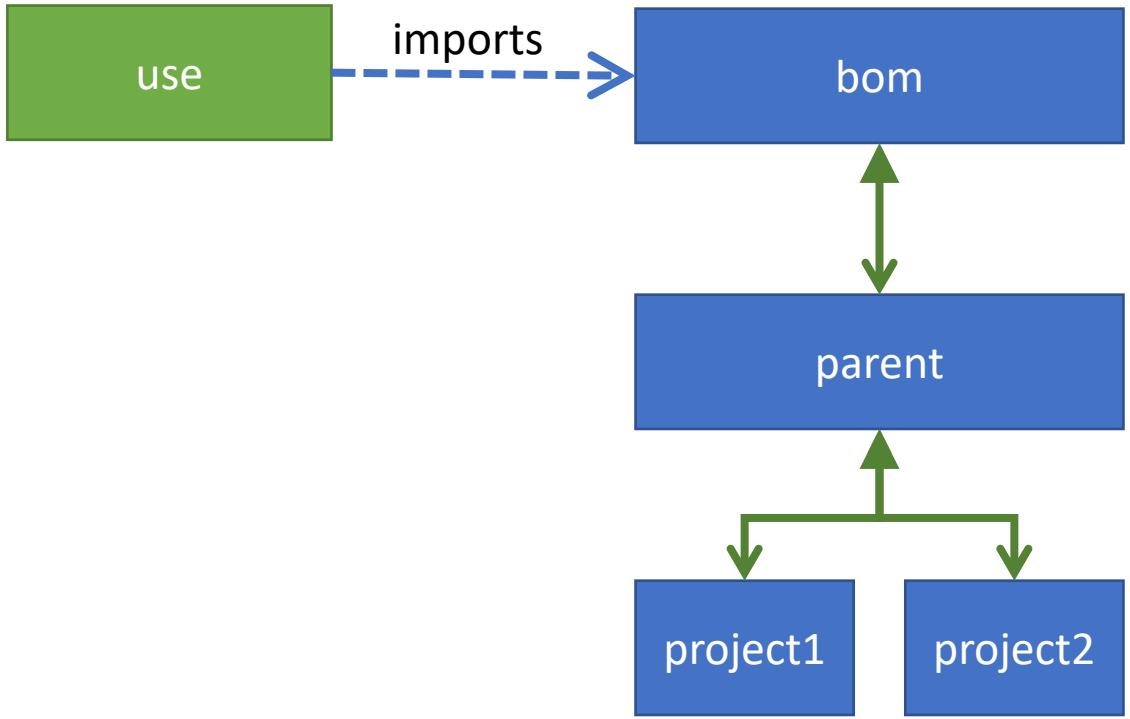
Bill of Materials (BOM) POMs

- The project that follows shows
  how the library can now be used in another project without having to specify the dependent project's versions.



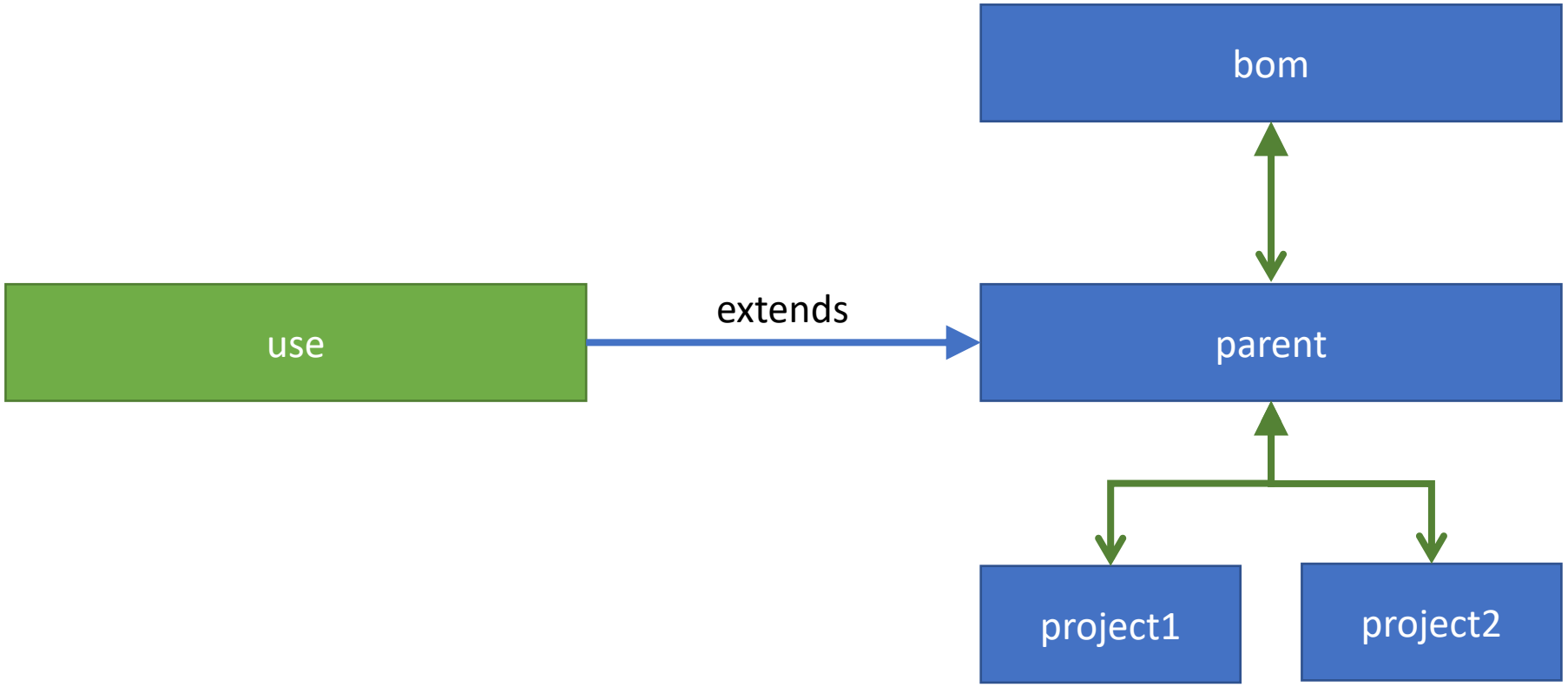
Bill of Materials (BOM) POMs

```
<project ... >

    <groupId>com.test</groupId>
    <artifactId>use</artifactId>
    <version>1.0.0</version>

    <dependencyManagement>
        <dependencies>
            <dependency>
                <groupId>com.test</groupId>
                <artifactId>bom</artifactId>
                <version>1.0.0</version>
                <type>pom</type>
                <scope>import</scope>
            </dependency>
        </dependencies>
    </dependencyManagement>

    <dependencies>
        <dependency>
            <groupId>com.test</groupId>
            <artifactId>project1</artifactId>
        </dependency>
        <dependency>
            <groupId>com.test</groupId>
            <artifactId>project2</artifactId>
        </dependency>
    </dependencies>

</project>
```



Bill of Materials (BOM) POMs

- The project that follows shows how the library can now be used in another project without having to specify the dependent project's versions.



Bill of Materials (BOM) POMs

```
<project ... >

    <groupId>com.test</groupId>
    <artifactId>use</artifactId>
    <version>1.0.0</version>

    <parent>
        <groupId>com.test</groupId>
        <artifactId>parent</artifactId>
        <version>1.0.0</version>
    </parent>

    <dependencies>
        <dependency>
            <groupId>com.test</groupId>
            <artifactId>project1</artifactId>
        </dependency>
        <dependency>
            <groupId>com.test</groupId>
            <artifactId>project2</artifactId>
        </dependency>
    </dependencies>

</project>
```